

题型

单项选择：每题2分，共30分。

名词解释：每题1分，共10分。

简答题：每题5分，共20分。

计算题：每题5分，共20分。

综合设计题：每题10分，共20分。

名词解释

1. ROM 只读存储器
2. CU 控制单元
3. DRAM 动态随机存取存储器
4. RISC 精简指令集计算机
5. EEPROM 电可擦除可编程只读存储器
6. HLDA 总线响应信号
7. MDR 存储器数据寄存器
8. INTA 中断响应信号
9. PC 程序计数器
10. EPROM 可编程只读存储器
11. MAR 存储器地址寄存器
12. DMA 直接内存访问
13. RAM 随机存取存储器
14. PROM 一次性编程只读存储器
15. ALU 算术逻辑单元
16. IR 指令寄存器
17. SRAM 静态随机存取存储器
18. CISC 复杂指令集计算机
19. MREQ 访存控制信号
20. FLOPS 每秒浮点运算次数

课件

一、冯·诺依曼计算机的特点

1. 计算机由五大部件组成
2. 指令和数据以同等地位存于存储器，可按地址寻访
3. 指令和数据用二进制表示
4. 指令由操作码和地址码组成
5. 指令在存储器内按顺序存放
6. 以运算器为中心

二、总线判优控制

基本概念：

- 主设备(模块)对总线有**控制权**
- 从设备(模块)**响应**从主设备发来的总线命令
- 总线判优控制：
- 集中式**（特别注意）**
 - 链式查询：特点：连线简单，易于扩充，对电路故障最敏感
 - 计数器定时查询：特点：优先级设置较灵活，对故障不敏感，连线及控制过程较复杂
 - 独立请求方式：特点：速度最快，但硬件器件用量大，连线多，成本较高
- 分布式

三、主存的技术指标

1.存储容量

主存：**存放**二进制代码的总位数

2.存储速度（注意）

- 存取时间：
 - 存储器的访问时间
 - 读出时间
 - 写入时间
- **存取周期（特别注意）：**
 - 连续两次独立的存储器操作（读或写）所需的 最小间隔时间 **（概念）**
 - 读周期
 - 写周期

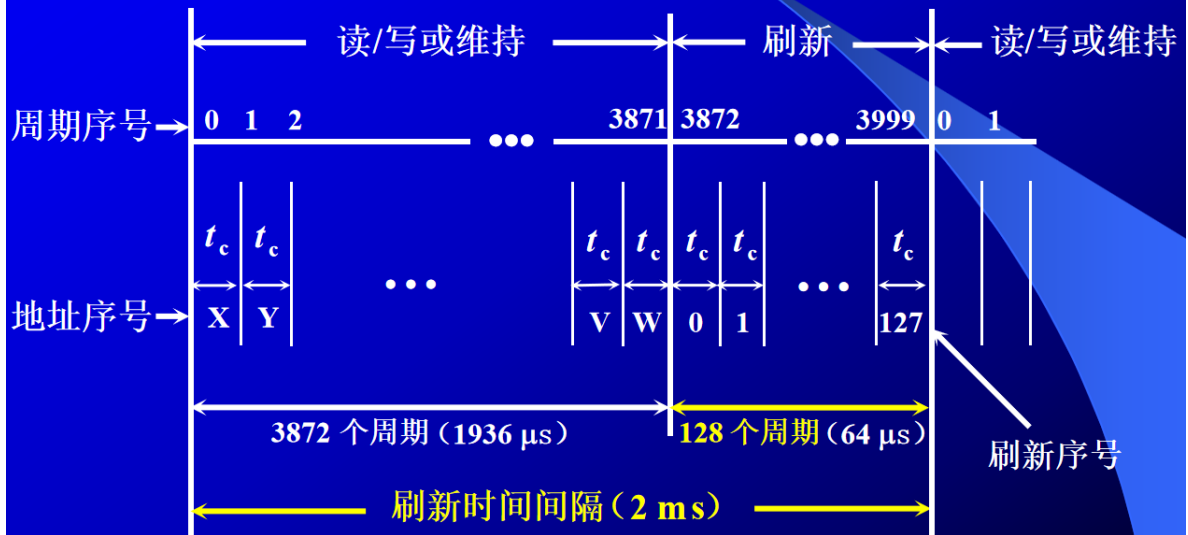
3.存储器的带宽：位/秒

四、动态RAM刷新（刷新与行地址有关）

简要说明三种形式和特点

1.集中刷新（存取周期为0.5 微秒）

① 集中刷新（存取周期为 $0.5\ \mu\text{s}$ ）以 128×128 矩阵为例



“死区” 为 $0.5\ \mu\text{s} \times 128 = 64\ \mu\text{s}$

“死时间率” 为 $128/4\ 000 \times 100\% = 3.2\%$

2.分散刷新（存取周期为1微秒）

② 分散刷新（存取周期为 $1\ \mu\text{s}$ ）

4.2

以 128×128 矩阵为例



$$t_C = t_M + t_R$$

无“死区”

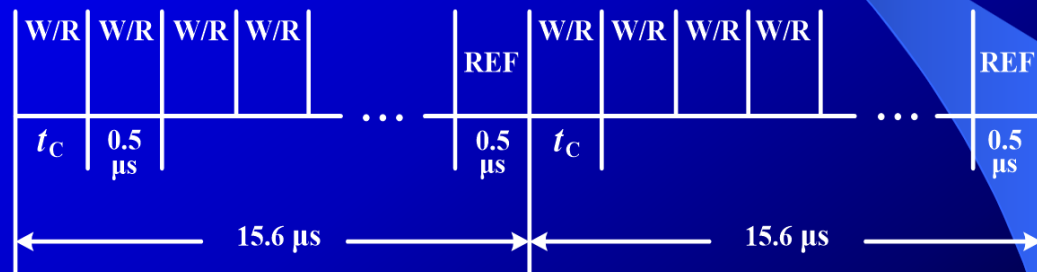
↓ ↓
读写 刷新

(存取周期为 $0.5\ \mu\text{s} + 0.5\ \mu\text{s}$)

3.分散刷新与集中刷新相结合（又被称为“异步刷新”）

③ 分散刷新与集中刷新相结合（异步刷新）^{4.4}

对于 128×128 的存储芯片（存取周期为 $0.5 \mu\text{s}$ ）
若每隔 $15.6 \mu\text{s}$ 刷新一次



每行每隔 2 ms 刷新一次 “死区” 为 $0.5 \mu\text{s}$

将刷新安排在指令译码阶段，不会出现“死区”

五、例题（4.1、4.2、4.3）（经常出大题）

大步骤：芯片的选择和数量、地址的解析、画图

4.1

例4.1 解：

(1) 写出对应的二进制地址码

A_{15}	A_{14}	A_{13}	A_{11}	A_{10}	...	A_7	...	A_4	A_3	...	A_0	
0	1	1	0	0	0	0	0	0	0	0	0	0
⋮												
0	1	1	0	0	1	1	1	1	1	1	1	1
0	1	1	0	1	0	0	0	0	0	0	0	0
⋮												
0	1	1	0	1	0	1	1	1	1	1	1	1

1片 $2\text{K} \times 8$ 位

ROM

$2\text{K} \times 8$ 位

$1\text{K} \times 8$ 位

RAM

2片 $1\text{K} \times 4$ 位

(2) 确定芯片的数量及类型

4.2

(3) 分配地址线

4.2

A_{15}	A_{13}	A_{11}	A_{10}	...	A_7	...	A_4	A_3	...	A_0	
0	1	1	0	0	0	0	0	0	0	0	1片 ROM $2K \times 8$ 位
...	...	...	...	...	...	...	...	...	...	...	
0	1	1	0	0	1	1	1	1	1	1	
0	1	1	0	1	0	0	0	0	0	0	2片 RAM $1K \times 4$ 位
...	...	...	...	...	...	...	...	...	...	...	
0	1	1	0	1	0	1	1	1	1	1	

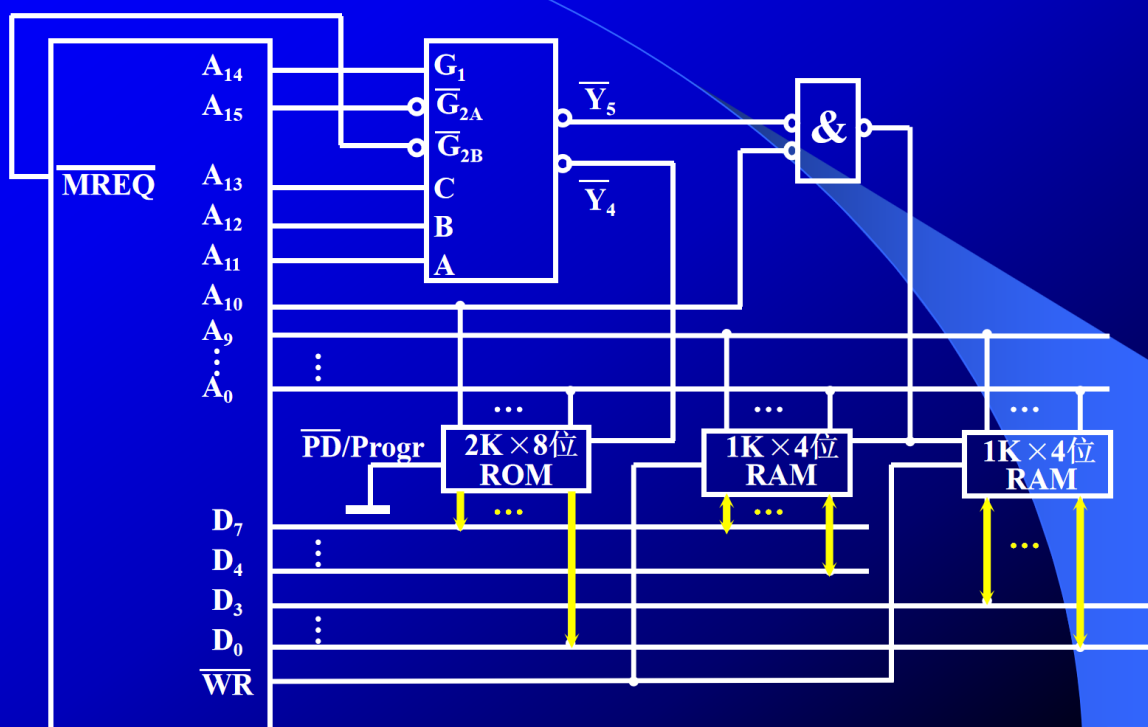
$A_{10} \sim A_0$ 接 $2K \times 8$ 位 ROM 的地址线

$A_9 \sim A_0$ 接 $1K \times 4$ 位 RAM 的地址线

(4) 确定片选信号

例 4.1 CPU 与存储器的连接图

4.2



例4.2 假设同前，要求最小 4K为系统程序区，相邻 8K为用户程序区。

(1) 写出对应的二进制地址码

(2) 确定芯片的数量及类型

1片 4K×8位 ROM **2片** 4K×8位 RAM

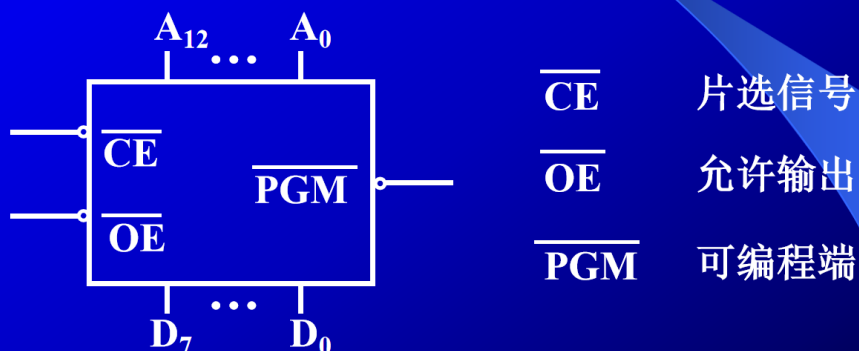
(3) 分配地址线

$A_{11} \sim A_0$ 接 ROM 和 RAM 的地址线

(4) 确定片选信号

4.3

例 4.3 设 CPU 有 20 根地址线，8 根数据线。并用 $\overline{IO/\overline{M}}$ 作访存控制信号。 $\overline{RD}$ 为读命令， $\overline{WR}$ 为写命令。现有 2764 EPROM (8K×8位)，外特性如下：



用 138 译码器及其他门电路（门电路自定）画出 CPU 和 2764 的连接图。要求地址为 F0000H~FFFFFH，并写出每片 2764 的地址范围。

教材例题（仅供参考）

以下是教材给的例题，这里给出仅供参考

在实际应用 CPU 与存储芯片时,还会遇到两者时序的配合、速度、负载匹配等问题,下面用一个实例来剖析 CPU 与存储芯片的连接方式。

例 4.1 设 CPU 有 16 根地址线、8 根数据线,并用 $\overline{\text{MREQ}}$ 作为访存控制信号(低电平有效),用 $\overline{\text{WR}}$ 作为读/写控制信号(高电平为读,低电平为写)。现有下列存储芯片:1 K×4 位 RAM、4 K×8 位 RAM、8 K×8 位 RAM、2 K×8 位 ROM、4 K×8 位 ROM、8 K×8 位 ROM 及 74138 译码器和各种门电路,如图 4.36 所示。画出 CPU 与存储器的连接图,要求如下:

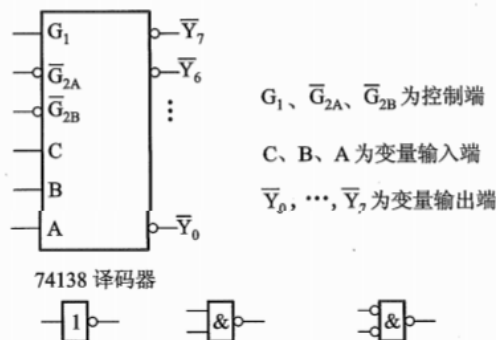


图 4.36 译码器和门电路

① 主存地址空间分配:

6000H~67FFH 为系统程序区。

6800H~6BFFH 为用户程序区。

② 合理选用上述存储芯片,说明各选几片。

③ 详细画出存储芯片的片选逻辑图。

解:第一步,先将十六进制地址范围写成二进制地址码,并确定其总容量。

A_{15}	$\cdots$	A_{12}	A_{11}	$\cdots$	A_8	A_7	$\cdots$	A_4	A_3	$\cdots$	A_0	
0	1	1	0	0	0	0	0	0	0	0	0	0
$\cdots$												
0	1	1	0	0	1	1	1	1	1	1	1	1
0	1	1	0	1	0	0	0	0	0	0	0	0
$\cdots$												
0	1	1	0	1	0	1	1	1	1	1	1	1

} 系统程序区
 } 2 K×8 位
 } 用户程序区
 } 1 K×8 位

第二步,根据地址范围的容量以及该范围在计算机系统中的作用,选择存储芯片。

根据 6000H~67FFH 为系统程序区的范围,应选择 1 片 2 K×8 位的 ROM,若选择 4 K×8 位或 8 K×8 位的 ROM,都超出了 2 K×8 位的系统程序区范围。

根据 6800H~6BFFH 为用户程序区的范围,选 2 片 1 K×4 位的 RAM 芯片正好满足 1 K×8 位的用户程序区要求。

第三步,分配 CPU 的地址线。

将 CPU 的低 11 位地址 $A_{10} \sim A_0$ 与 2 K×8 位的 ROM 地址线相连;将 CPU 的低 10 位地址 $A_9 \sim A_0$ 与 2 片 1 K×4 位的 RAM 地址线相连。剩下的高位地址与访存控制信号 $\overline{\text{MREQ}}$ 共同产生存储芯片的片选信号。

第四步,片选信号的形成。

由图 4.36 给出的 74138 译码器输入逻辑关系可知,必须保证控制端 G_1 为高电平, $\overline{G_{2A}}$ 与 $\overline{G_{2B}}$ 为低电平,才能使译码器正常工作。根据第一步写出的存储器地址范围得出, A_{15} 始终为低电平, A_{14} 始终为高电平,它们正好可分别与译码器的 $\overline{G_{2A}}$ (低)和 G_1 (高)对应。而访存控制信号 $\overline{\text{MREQ}}$ (低电平有效)又正好可与 $\overline{G_{2B}}$ (低)对应。剩下的 A_{13} 、 A_{12} 、 A_{11} 可分别接到译码器的 C、B、A 输入端。其输出 $\overline{Y_4}$ 有效时,选中 1 片 ROM; $\overline{Y_5}$ 与 A_{10} 同时有效均为低电平时,与门输出选中 2 片 RAM,如图 4.37 所示。图中 ROM 芯片的 $\overline{\text{PD/Progr}}$ 端接地,以确保在读出时低电平有效。RAM 芯片的读写控制端与 CPU 的读写命令端 $\overline{\text{WR}}$ 相连。ROM 的 8 根数据线直接与 CPU 的 8 根数据线相连,2 片 RAM 的数据线分别与 CPU 数据总线的高 4 位和低 4 位相连。

例 4.2 假设 CPU 及其他芯片同例 4.1,画出 CPU 与存储器的连接图。要求主存的地址空间满足下述条件:最小 8 K 地址为系统程序区,与其相邻的 16 K 地址为用户程序区,最大 4 K 地址空间为系统程序工作区。详细画出存储芯片的片选逻辑并指出存储芯片的种类及片数。

解:第一步,根据题目的地址范围写出相应的二进制地址码。

A_{15}	$\cdots$	A_{12}	A_{11}	$\cdots$	A_8	A_7	$\cdots$	A_4	A_3	$\cdots$	A_0	
1	1	1	1	0	0	0	0	0	0	0	0	0
$\cdots$												
1	1	1	1	1	1	1	1	1	1	1	1	1

最大 4 K×8 位

系统程序工作

第二步,根据地址范围的容量及其在计算机系统中的作用,确定最小 8 KB 系统程序区选择 1 片 8 K×8 位 ROM;与其相邻的 16 KB 用户程序区选择 2 片 8 K×8 位 RAM;最大 4 KB 系统程序工作区选择 1 片 4 K×8 位 RAM。

第三步,分配 CPU 地址线。

将 CPU 的低 13 位地址线 $A_{12} \sim A_0$ 与 1 片 8 K×8 位 ROM 和 2 片 8 K×8 位 RAM 的地址线相连;将 CPU 的低 12 位地址线 $A_{11} \sim A_0$ 与 1 片 4 K×8 位 RAM 的地址线相连。

第四步,形成片选信号。

将 74138 译码器的控制端 G_1 接 +5V, $\overline{G_{2A}}$ 和 $\overline{G_{2B}}$ 接 $\overline{MREQ}$,以保证译码器正常工作。CPU 的 $A_{15}A_{14}A_{13}$ 分别接在译码器的 C、B、A 端,作为变量输入,则其输出 $\overline{Y_0}$ 、 $\overline{Y_1}$ 、 $\overline{Y_2}$ 分别作为 ROM、 RAM_1 和 RAM_2 的片选信号。此外,根据题意,最大 4 K 地址范围的 A_{12} 为高电平,故经反相后再与 $\overline{Y_7}$ 相“与”,其输出作为 4 K×8 位 RAM 的片选信号,如图 4.38 所示。

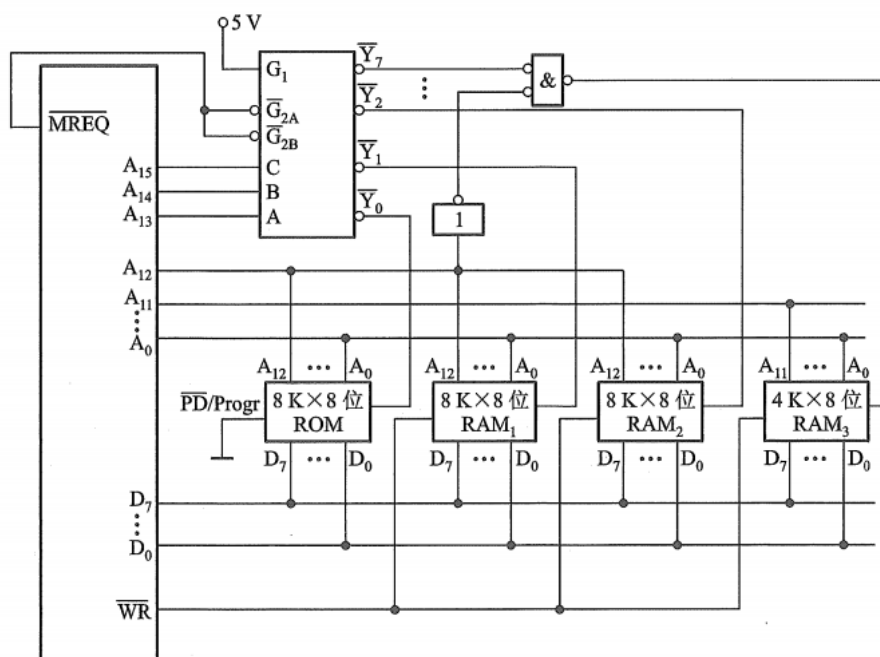


图 4.38 例 4.2 CPU 与存储芯片的连接图

例 4.3 设 CPU 有 20 根地址线和 16 根数据线,并用 $\overline{IO/\overline{M}}$ 作为访存控制信号, $\overline{RD}$ 为读命

令, $\overline{WR}$ 为写命令。CPU 可通过 BHE 和 A_0 来控制按字节或字两种形式访存(如表 4.1 所示)。要求采用图 4.39 所示的芯片,门电路自定。试回答:

- (1) CPU 按字节访问和按字访问的地址范围各是多少?
- (2) CPU 按字节访问时需分奇偶体,且最大 64 KB 为系统程序区,与其相邻的 64 KB 为用户程序区。写出每片存储芯片所对应的二进制地址码。
- (3) 画出对应上述地址范围的 CPU 与存储芯片的连接图。

表 4.1 例 4.3 CPU 访问形式与 BHE 和 A_0 的关系

BHE	A_0	访问形式
0	0	字
0	1	奇字节
1	0	偶字节
1	1	不访问

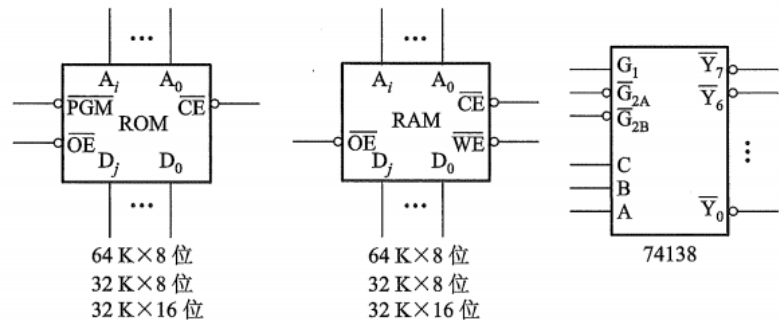


图 4.39 例 4.3 芯片

解:(1) CPU 按字节访问的地址范围为 1 M,CPU 按字访问的地址范围是 512 K。
 (2) 由于 CPU 按字节访存时需区分奇偶体,并且还可以按字访问,因此如果选择 64 K×8 位的芯片,按字节访问时体现不出奇偶分体;如果选择 32 K×16 位的芯片,虽然能按字访问,但不能满足以字节为最小单位。故一律选择 32 K×8 位的存储芯片,其中系统程序区 64 KB 选择 2 片 32 K×8 位 ROM,用户程序区 64 KB 选择 2 片 32 K×8 位 RAM。它们对应的二进制地址范围如下:

$$\begin{array}{cccccccccccccccccccc}
 A_{19} & \cdots & A_{16} & A_{15} & \cdots & A_{12} & A_{11} & \cdots & A_8 & A_7 & \cdots & A_4 & A_3 & \cdots & A_0 & & \\
 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & \\
 \cdots & & & & & & & & & & & & & & & & & \\
 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0
 \end{array}
 \left. \begin{array}{l}
 64\text{ K}\times 8\text{ 位 ROM,} \\
 \text{其中 1 片 32 K}\times 8\text{ 位(奇)} \\
 \text{1 片 32 K}\times 8\text{ 位(偶)}
 \end{array} \right\}$$

$A_{19} \cdots A_{16}$	$A_{15} \cdots A_{12}$	$A_{11} \cdots A_8$	$A_7 \cdots A_4$	$A_3 \cdots A_0$	
1 1 1 0	1 1 1 1	1 1 1 1	1 1 1 1	1 1 1 1	} 64 K×8 位 RAM, 其中 1 片 32 K×8 位(奇) 1 片 32 K×8 位(偶)
...					
1 1 1 0	0 0 0 0	0 0 0 0	0 0 0 0	0 0 0 0	

该题的难点在于片选逻辑。由于 CPU 按字访问还是按字节访问受 BHE 和 A_0 的控制,因此可用 BHE 和 A_0 分别控制 74138 译码器的输入端 B 和 A,而 $A_{15} \sim A_1$ 与存储芯片的地址线相连,余下的 A_{16} 接 74138 译码器的输入端 C。 A_{19} 、 A_{18} 、 A_{17} 作为与门的输入端,与门输出接至 74138 译码器的 G_1 端, $\overline{G_{2A}}$ 和 $\overline{G_{2B}}$ 与 $IO/\overline{M}$ 相连,以确保 74138 译码器正常工作。具体连接图如图 4.40 所示。

图中译码器输出 $\overline{Y}_4$ 有效时,同时选择 ROM_1 和 ROM_2 , CPU 以字形式访问; $\overline{Y}_5$ 有效时选择 ROM_1 (奇体), $\overline{Y}_6$ 有效时选择 ROM_2 (偶体), CPU 以字节形式访问。同理,译码器输出 $\overline{Y}_0$ 控制 CPU 可按字节形式访问 RAM_1 和 RAM_2 。 $\overline{Y}_1$ 和 $\overline{Y}_2$ 分别按字节访问 RAM_1 (奇体)和 RAM_2 (偶体)。CPU 的读命令 $\overline{RD}$ 直接和 ROM、RAM 的 $\overline{OE}$ (允许输出端)相连, CPU 的写命令 $\overline{WR}$ 直接和 RAM 芯片的 $\overline{WE}$ (允许写输入端)相连。ROM 芯片的 $\overline{PGM}$ 端低电平时可编程,接高电平 V_{CC} 时可按只读方式工作, $\overline{CE}$ 为片信号,分别与不同的译码输出端相连。

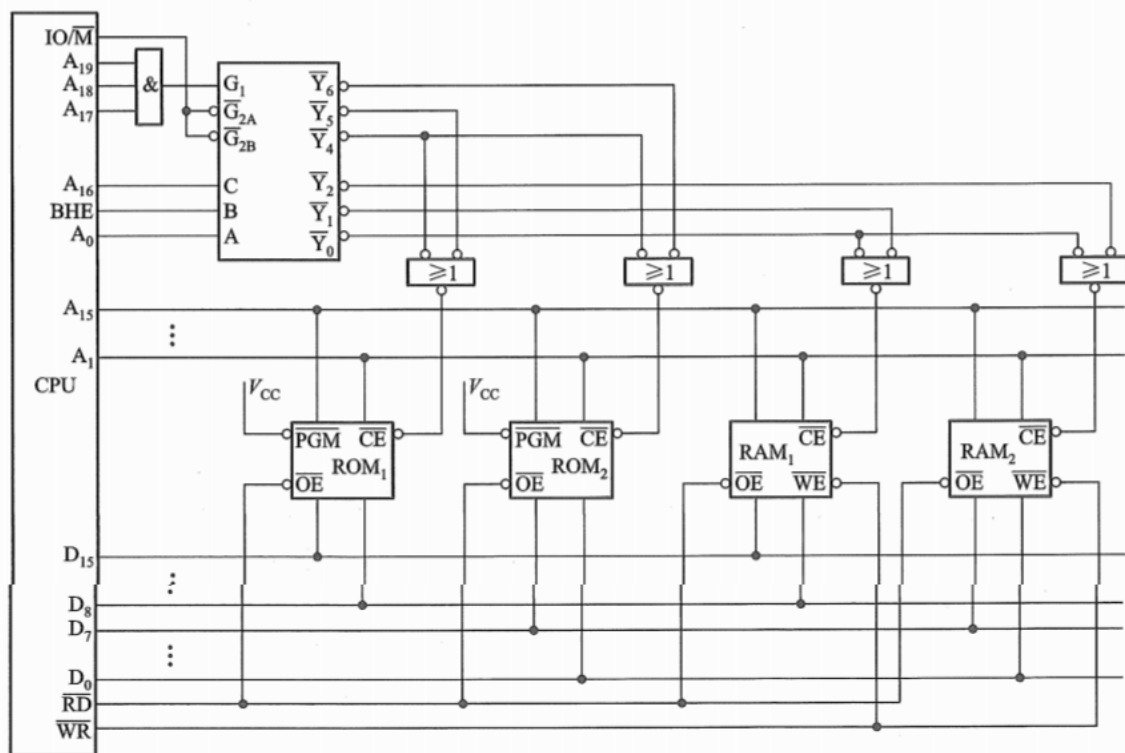


图 4.40 例 4.3 CPU 与存储芯片的连接图

六、奇校验和偶校验

奇校验：二进制比特序列中（包括校验位），1 的个数为奇数就为奇校验

偶校验：二进制比特序列中（包括校验位），1 的个数为偶数就为偶校验

举个例子：

假设你要传输一个 8 位数据： 10110010

表格 复制			
校验方式	数据中1的个数	校验位应设为	最终传输的9位数据
奇校验	4 (偶数)	1	110110010
偶校验	4 (偶数)	0	010110010

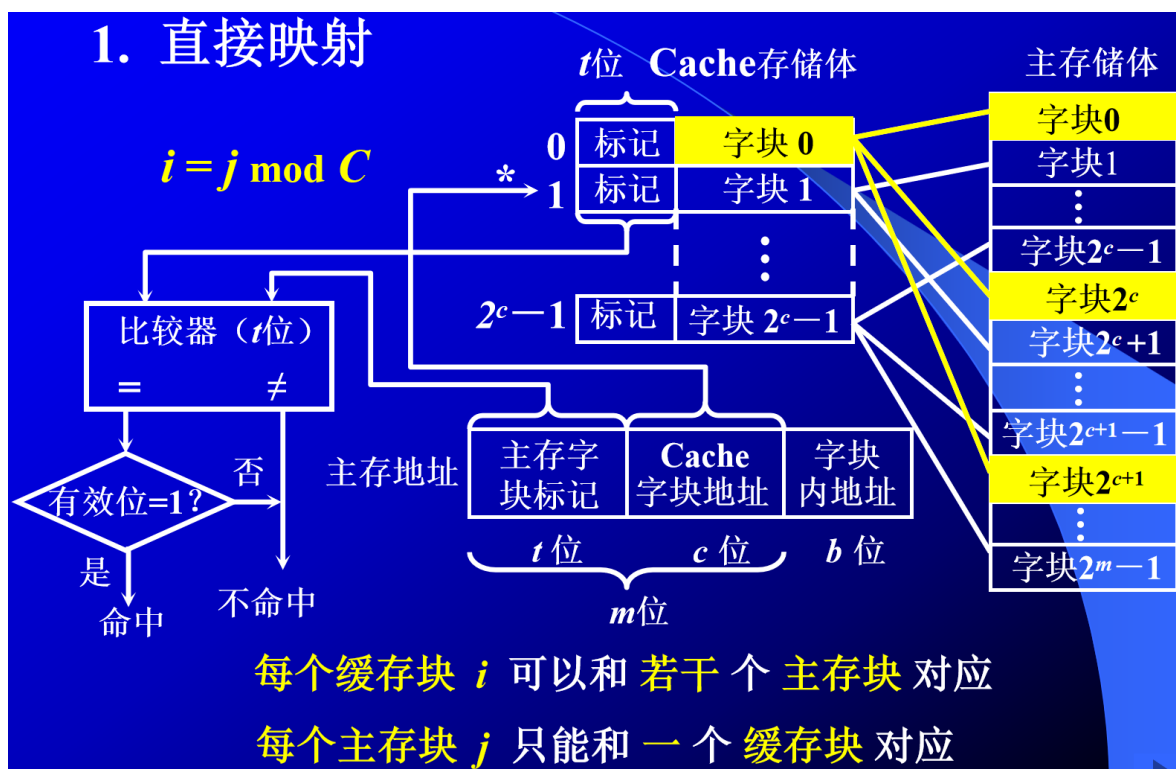
七、Cache - 主存的地址映射（经常出大题）（必须掌握）

有关三种映射的计算必须掌握

地址格式也要掌握

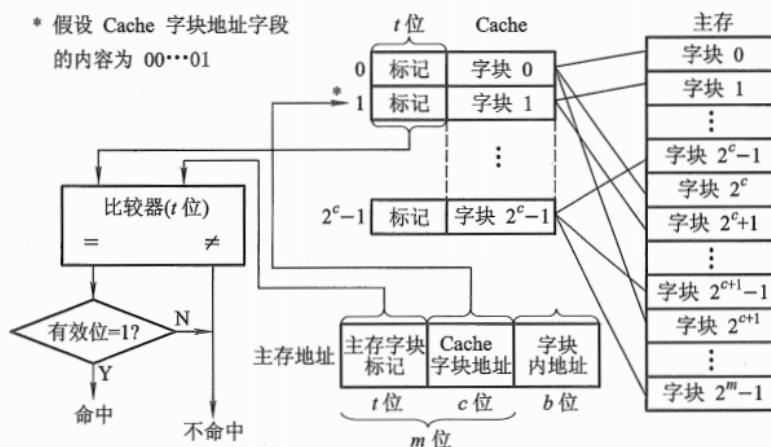
原理复杂但做题简单

1.直接映射



1. 直接映射

图 4.54 示出了直接映射方式主存与缓存中字块的对应关系。



图中每个主存块只与一个缓存块相对应,映射关系式为

$$i = j \bmod C \quad \text{或} \quad i = j \bmod 2^c$$

其中, i 为缓存块号, j 为主存块号, C 为缓存块数。映射结果表明每个缓存块对应若干个主存块,如表 4.4 所示。

表 4.4 直接映射方式主存块和缓存块的对应关系

缓存块	主存块
0	$0, C, \dots, 2^m - C$
1	$1, C + 1, \dots, 2^m - C + 1$
...	...
$C - 1$	$C - 1, 2C - 1, \dots, 2^m - 1$

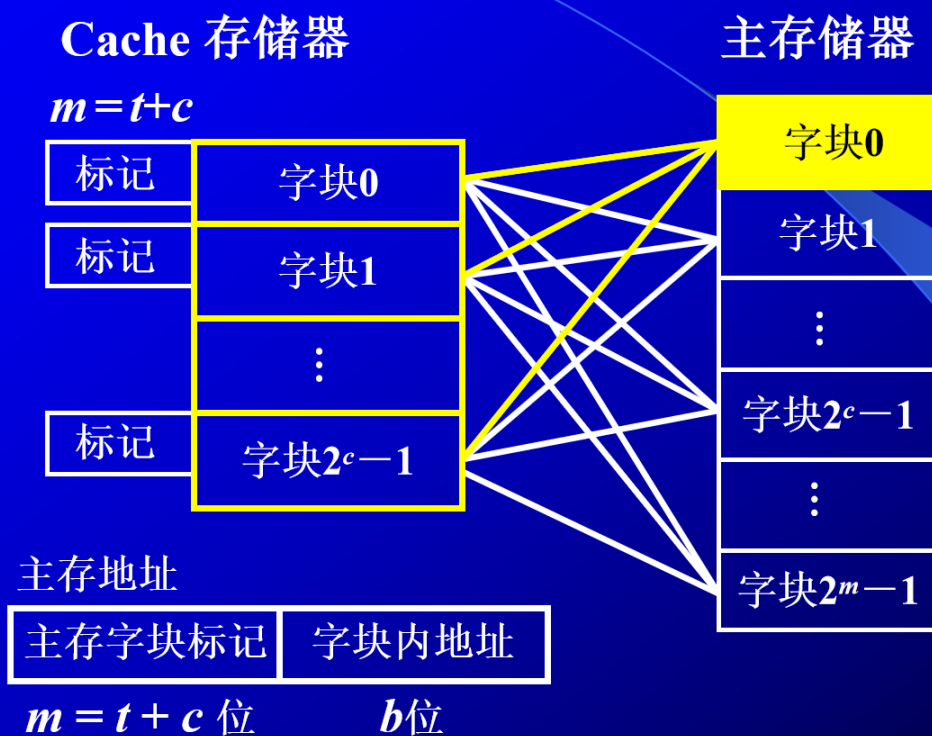
这种方式的优点是实现简单,只需利用主存地址的某些位直接判断,即可确定所需字块是否在缓存中。由图 4.54 可见,主存地址高 m 位被分成两部分:低 c 位是指 Cache 的字块地址,高 t 位($t = m - c$)是指主存字块标记,它被记录在建立了对应关系的缓存块的“标记”位中。当缓存接

到 CPU 送来的主存地址后,只需根据中间 c 位字段(假设为 00...01)找到 Cache 字块 1,然后根据字块 1 的“标记”是否与主存地址的高 t 位相符来判断,若符合且有效位为“1”(有效位用来识别 Cache 存储块中的数据是否有效,因为有时 Cache 中的数据是无效的,例如,在初始时刻 Cache 应该是“空”的,其中的内容是无意义的),表示该 Cache 块已和主存的某块建立了对应关系(即已命中),则可根据 b 位地址从 Cache 中取得信息;若不符合,或有效位为“0”(即不命中),则从主存读入新的字块来替代旧的字块,同时将信息送往 CPU,并修改 Cache“标记”。如果原来有效位为“0”,还得将有效位置成“1”。

直接映射方式的缺点是不够灵活,因每个主存块只能固定地对应某个缓存块,即使缓存内还空着许多位置也不能占用,使缓存的存储空间得不到充分的利用。此外,如果程序恰好要重复访问对应同一缓存位置的不同主存块,就要不停地替换,从而降低命中率。

2. 全相联映射

2. 全相联映射



主存中的任一块可以映射到缓存中的任一块

2. 全相联映射

全相联映射允许主存中每一字块映射到 Cache 中的任何一块位置上,如图 4.55 所示。这种映射方式可以从已被占满的 Cache 中替换出任一旧字块。显然,这种方式灵活,命中率也更高,缩小了块冲突率。与直接映射相比,它的主存字块标记从 t 位增加到 $t+c$ 位,这就使 Cache“标记”的位数增多,而且访问 Cache 时主存字块标记需要和 Cache 的全部“标记”位进行比较,才能判断出所访问主存地址的内容是否已在 Cache 内。这种比较通常采用“按内容寻址”的相联存储器(见附录 4A)来完成。

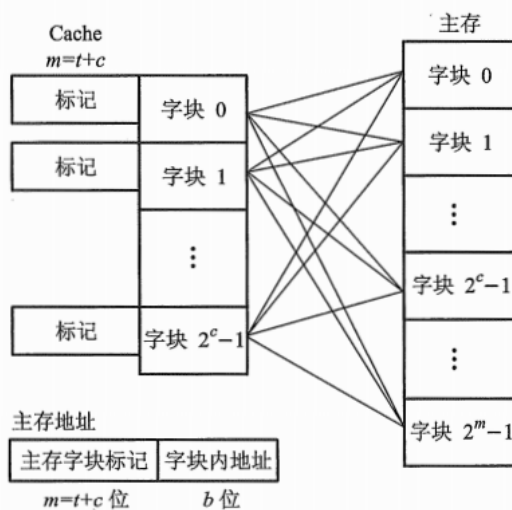


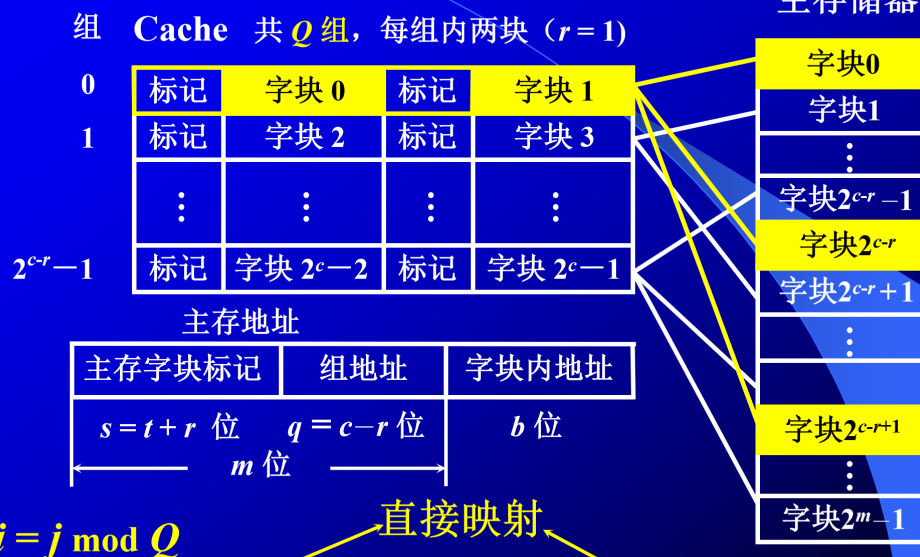
图 4.55 全相联映射

总之,这种方式所需的逻辑电路甚多,成本较高,实际的 Cache 还要采用各种措施来减少地址的比较次数。

3. 组相联映射

3. 组相联映射

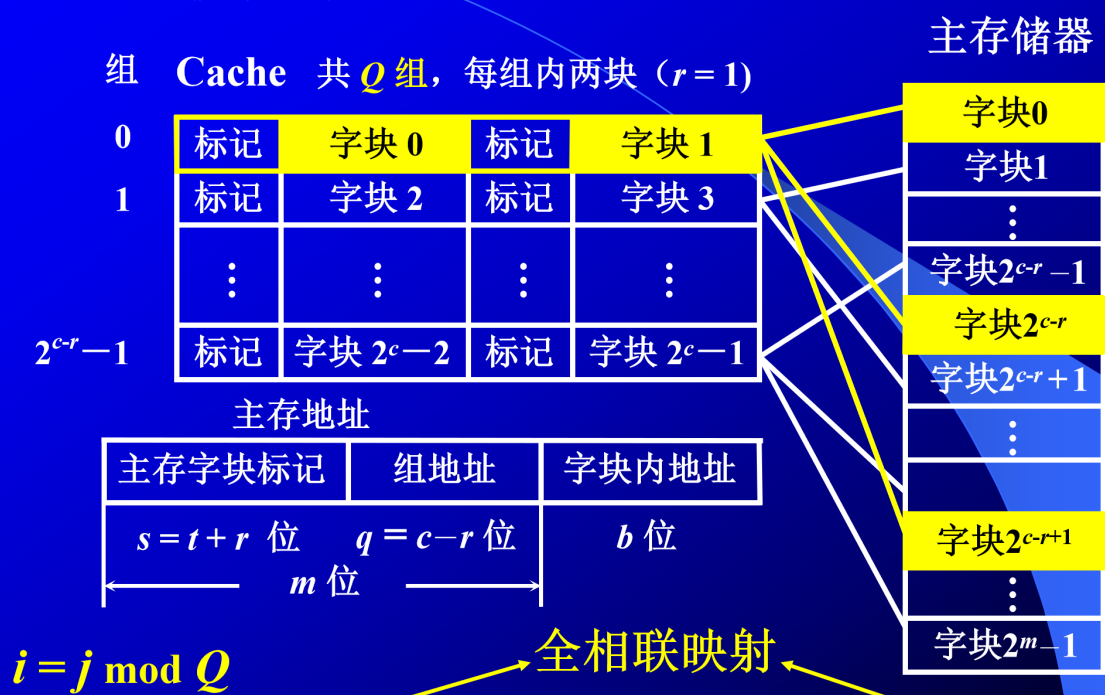
4.3



某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

3. 组相联映射

4.3



某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

3. 组相联映射

4.3

组 Cache 共 Q 组，每组内两块 ($r=1$)

0	标记	字块 0	标记	字块 1
1	标记	字块 2	标记	字块 3
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$2^{c-r}-1$	标记	字块 2^c-2	标记	字块 2^c-1

主存地址

主存字块标记	组地址	字块内地址
$s = t + r$ 位	$q = c - r$ 位	b 位
m 位		

主存储器

字块0
字块1
$\vdots$
字块 $2^{c-r}-1$
字块 2^{c-r}
字块 $2^{c-r}+1$
$\vdots$
字块 $2^{c-r}+1$
$\vdots$
字块 2^m-1

$$i = j \bmod Q$$

全相联映射

某一主存块 j 按模 Q 映射到 缓存 的第 i 组中的 任一块

3. 组相联映射

组相联映射是对直接映射和全相联映射的一种折中。它把 Cache 分为 Q 组, 每组有 R 块, 并有以下关系:

$$i = j \bmod Q$$

其中, i 为缓存的组号, j 为主存的块号。某一主存块按模 Q 将其映射到缓存的第 i 组内, 如图 4.56 所示。

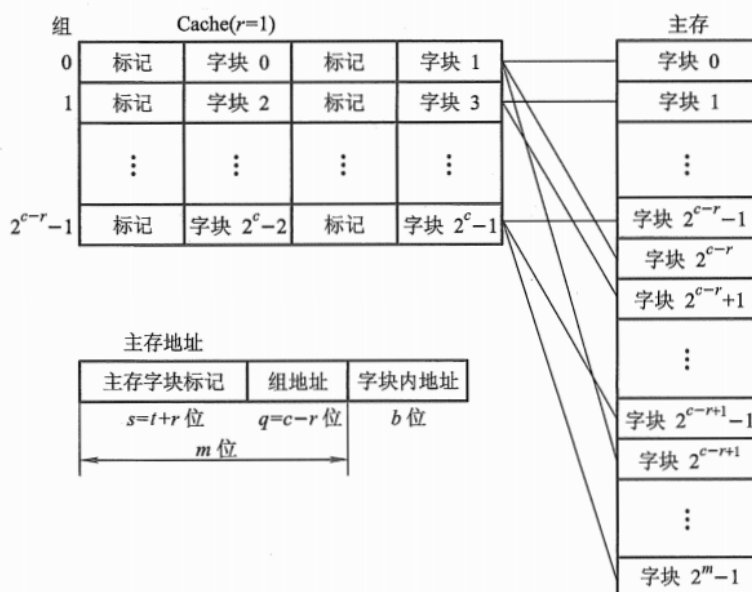


图 4.56 组相联映射

组相联映射的主存地址各段与直接映射(参见图 4.54)相比,还是有区别的。图 4.54 中 Cache 字块地址字段由 c 位变为组地址字段 q 位,且 $q=c-r$,其中 2^c 表示 Cache 的总块数, 2^q 表示 Cache 的分组个数, 2^r 表示组内包含的块数。主存字块标记字段由 t 位变为 $s=t+r$ 位。为了便于理解,假设 $c=5, q=4$, 则 $r=c-q=1$ 。其实际含义为:Cache 共有 $2^c=32$ 个字块,共分为 $2^q=16$ 组,每组内包含 $2^r=2$ 块。组内 2 块的组相联映射又称为二路组相联。

根据上述假设条件,组相联映射的含义是:主存的某一字块可以按模 16 映射到 Cache 某组的任一字块中。即主存的第 0, 16, 32...字块可以映射到 Cache 第 0 组 2 个字块中的任一字块;主存的第 15, 31, 47...字块可以映射到 Cache 第 15 组中的任一字块。显然,主存的第 j 块会映射到 Cache 的第 i 组内,两者之间一一对应,属直接映射关系;另一方面,主存的第 j 块可以映射到 Cache 的第 i 组内中的任一块,这又体现出全相联映射关系。可见,组相联映射的性能及其复杂性介于直接映射和全相联映射两者之间,当 $r=0$ 时是直接映射方式,当 $r=c$ 时是全相联映射方式。

教材例题

例 4.8 假设主存容量为 512 KB,Cache 容量为 4 KB,每个字块为 16 个字,每个字 32 位。

(1) Cache 地址有多少位?可容纳多少块?

(2) 主存地址有多少位?可容纳多少块?

(3) 在直接映射方式下,主存的第几块映射到 Cache 中的第 5 块(设起始字块为第 1 块)?

(4) 画出直接映射方式下主存地址字段中各段的位数。

解:(1) 根据 Cache 容量为 4 KB($2^{12}=4\text{ K}$),Cache 地址为 12 位。由于每字 32 位,则 Cache 共有 $4\text{ KB}/4\text{ B}=1\text{ K}$ 字。因每个字块 16 个字,故 Cache 中有 $1\text{ K}/16=64$ 块。

(2) 根据主存容量为 512 KB($2^{19}=512\text{ K}$),主存地址为 19 位。由于每字 32 位,则主存共有 $512\text{ KB}/4\text{ B}=128\text{ K}$ 字。因每个字块 16 个字,故主存中共 $128\text{ K}/16=8\text{ 192}$ 块。

(3) 在直接映射方式下,由于 Cache 共有 64 块,主存共有 8 192 块,因此主存的 $5,64+5,2\times 64+5,\dots,2^{13}-64+5$ 块能映射到 Cache 的第 5 块中。

(4) 在直接映射方式下,主存地址字段的各段位数分配如图 4.57 所示。其中字块内地址为 6 位(4 位表示 16 个字,2 位表示每字 32 位),缓存共 64 块,故缓存字块地址为 6 位,主存字块标记记为主存地址长度与 Cache 地址长度之差,即 $19-12=7$ 位。

主存字块标记	缓存字块地址	字块内地址
7 位	6 位	6 位

图 4.57 例 4.8 主存地址各字段的分配

例 4.9 假设主存容量为 $512\text{ K}\times 16$ 位,Cache 容量为 $4\text{ 096}\times 16$ 位,块长为 4 个 16 位的字,访存地址为字地址。

(1) 在直接映射方式下,设计主存的地址格式。

(2) 在全相联映射方式下,设计主存的地址格式。

(3) 在二路组相联映射方式下,设计主存的地址格式。

(4) 若主存容量为 $512\text{ K}\times 32$ 位,块长不变,在四路组相联映射方式下,设计主存的地址格式。

解:(1) 根据 Cache 容量为 $4\text{ 096}=2^{12}$ 字,得 Cache 字地址为 12 位。根据块长为 4,且访存地址为字地址,得字块内地址为 2 位,即 $b=2$,且 Cache 共有 $4\text{ 096}/4=1\text{ 024}=2^{10}$ 块,即 $c=10$ 。根据主存容量为 $512\text{ K}=2^{19}$ 字,得主存字地址为 19 位。在直接映射方式下,主存字块标记为 $19-12=7$ 。主存的地址格式如图 4.58(a) 所示。

(2) 在全相联映射方式下,主存字块标记为 $19-b=19-2=17$ 位,其地址格式如图 4.58(b) 所示。

(3) 根据二路组相联的条件,一组内有 2 块,得 Cache 共分 $1\text{ 024}/2=512=2^9$ 组,即 $q=9$,主存字块标记为 $19-q-b=19-9-2=8$ 位,其地址格式如图 4.58(c) 所示。

(4) 若主存容量改为 $512\text{ K}\times 32$ 位,即双字宽存储器,块长仍为 4 个 16 位的字,访存地址仍为字地址,则主存容量可写为 $1\,024\text{ K}\times 16$ 位,得主存地址为 20 位。由四路组相联,得 Cache 共分 $1\,024/4=256=2^8$ 组,即 $q=8$ 。对应该条件下,主存字块标记为 $20-8-2=10$ 位,其地址格式如图 4.58(d)所示。

主存字块标记	Cache 字块地址	字块内地址
7	10	2

(a) 直接映射方式主存地址格式

主存字块标记	字块内地址
17	2

(b) 全相联映射方式主存地址格式

主存字块标记	组地址	字块内地址
8	9	2

(c) 二路组相联映射方式主存地址格式

主存字块标记	组地址	字块内地址
10	8	2

(d) 四路组相联映射方式双字宽主存地址格式

图 4.58 例 4.9 主存地址格式

例 4.11 设某机主存容量为 16 MB,Cache 的容量为 8 KB。每字块有 8 个字,每字 32 位。设计一个四路组相联映射的 Cache 组织。

(1) 画出主存地址字段中各段的位数。

(2) 设 Cache 初态为空,CPU 依次从主存第 0,1,2,⋯,99 号单元读出 100 个字(主存一次读出一个字),并重复此次序读 10 次,问命中率是多少?

(3) 若 Cache 的速度是主存速度的 5 倍,试问有 Cache 和无 Cache 相比,速度提高多少倍?

(4) 系统的效率为多少?

解:(1) 根据每个字块有 8 个字,每个字 32 位,得出主存地址字段中字块内地址字段为 5 位,其中 3 位为字地址,2 位为字节地址。

根据 Cache 容量为 $8\text{ KB} = 2^{13}\text{ B}$,字块大小为 2^5 B ,得 Cache 共有 2^8 块,故 $c = 8$ 。根据四路组相联映射 $2^r = 4$,得 $r = 2$,则 $q = c - r = 8 - 2 = 6$ 位。

根据主存容量为 $16\text{ MB} = 2^{24}\text{ B}$,得出主存地址字段中主存字块标记为 $24 - 6 - 5 = 13$ 位。

主存地址字段各段格式如图 4.59 所示。

主存字块标记	组地址	字块内地址
13 位	6 位	5 位

图 4.59 例 4.10 主存地址字段

(2) 由于每个字块中有 8 个字,而且初态 Cache 为空,因此 CPU 读第 0 号单元时,未命中,必须访问主存,同时将该字所在的主存块调入 Cache 第 0 组中的任一块内,接着 CPU 读 1~7 号单元时均命中。同理,CPU 读第 8,16,⋯,96 号单元时均未命中。可见 CPU 在连续读 100 个字中共有 13 次未命中,而后 9 次循环读 100 个字全部命中,命中率为

$$\frac{100 \times 10 - 13}{100 \times 10} = 0.987$$

(3) 根据题意,设主存取周期为 $5t$,Cache 的存取周期为 t ,没有 Cache 的访问时间为 $5t \times 1\,000$,有 Cache 的访问时间为 $t(1\,000 - 13) + 5t \times 13$,则有 Cache 和没有 Cache 相比,速度提高的倍数为

$$\frac{5t \times 1\,000}{t(1\,000 - 13) + 5t \times 13} - 1 \approx 3.75$$

(4) 根据(2)求得的命中率 0.987,主存的存取周期为 $5t$,Cache 的存取周期为 t ,得系统的效率为

$$\frac{t}{0.987 \times t + (1 - 0.987) \times 5t} \times 100\% = 95\%$$

八、输入输出系统获取数据的三种方式

1. 程序查询方式

特点和应用场景: 温度、湿度传感器等实时性要求不高的传感器,一般采用程序查询方式

2. 程序中断方式

特点和应用场景: 键盘鼠标等实时性要求高的传感器,一般采用程序中断方式

3. DMA方式

特点和应用场景: 硬盘等通常需要大量数据和主存进行数据交换的设备或实时性要求高的高速传感器,一般采用DMA方式

九、I/O 设备与主机的联系方式

1. I/O 设备编址方式 (主要注意的部分)

(1) 统一编址 用取数、存数指令

特点：I/O地址采用与主存单元地址完全一样的格式，I/O设备和主存占用同一个地址空间，CPU可像访问主存一样访问I/O设备，不需要安排专门的I/O指令

(2) 不统一编址（独立编制） 有专门的 I/O 指令

特点：机器为I/O设备专门安排一套完全不同于主存地址格式的地址编码，此时I/O地址与主存地址是两个独立的空间，CPU需要通过专门的I/O指令来访问I/O地址空间

2.设备选址

用设备选择电路识别是否被选中

3.传送方式

(1) 串行

(2) 并行

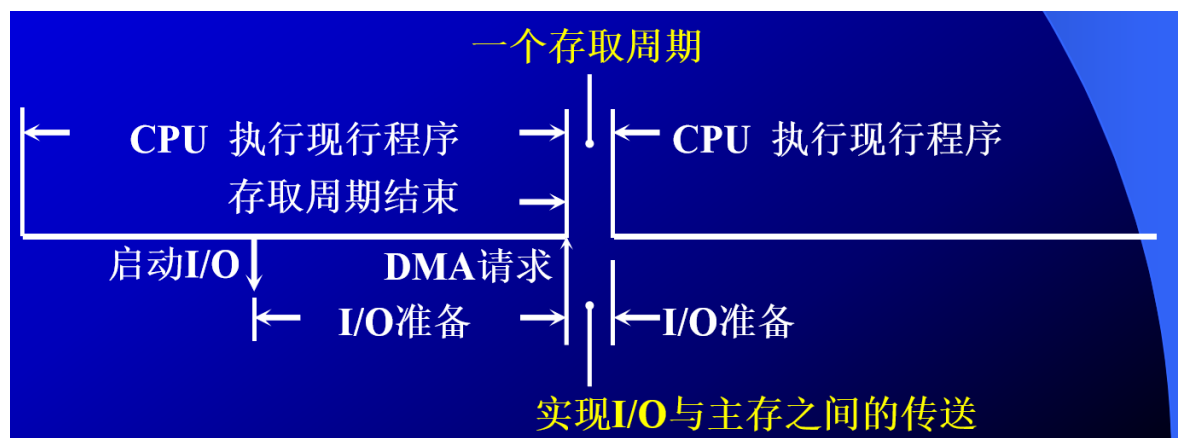
十、DMA 方式（DMA的工作特点）

1.主存和 I/O 之间有一条直接数据通道

2.不中断现行程序

3.周期挪用（周期窃取）

4.CPU 和 I/O 并行工作



十一、对原码、补码、反码等计算

这里方便复习，把各个概念系统地展示一下

1.无符号数

一、无符号数

寄存器的位数

反映无符号数的表示范围



8 位

0 ~ 255



16 位

0 ~ 65535

2.有符号数

2.1.机器数与真值

真值

带符号的数

+ 0.1011

- 0.1011

+ 1100

- 1100

机器数

符号数字化的数



小数点的位置

小数点的位置

小数点的位置

小数点的位置

2.2.原码表示法

原码的特点：简单、直观

(1) 定义

整数

$$[x]_{\text{原}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

x 为真值 n 为整数的位数

如 $x = +1110$ $[x]_{\text{原}} = 0, 1110$ 用 **逗号** 将符号位和数值部分隔开

$x = -1110$ $[x]_{\text{原}} = 2^4 + 1110 = 1, 1110$

带符号的绝对值表示

小数

6.1

$$[x]_{\text{原}} = \begin{cases} x & 1 > x \geq 0 \\ 1 - x & 0 \geq x > -1 \end{cases}$$

x 为真值

如 $x = +0.1101$ $[x]_{\text{原}} = 0.1101$ 用 **小数点** 将符号位和数值部分隔开

$x = -0.1101$ $[x]_{\text{原}} = 1 - (-0.1101) = 1.1101$

$x = +0.1000000$ $[x]_{\text{原}} = 0.1000000$ 用 **小数点** 将符号位和数值部分隔开

$x = -0.1000000$ $[x]_{\text{原}} = 1 - (-0.1000000) = 1.1000000$

(2) 举例

例 6.1 已知 $[x]_{\text{原}} = 1.0011$ 求 x $- 0.0011$

解：由定义得

$$x = 1 - [x]_{\text{原}} = 1 - 1.0011 = -0.0011$$

例 6.2 已知 $[x]_{\text{原}} = 1,1100$ 求 x -1100

解：由定义得

$$x = 2^4 - [x]_{\text{原}} = 10000 - 1,1100 = -1100$$

例 6.3 已知 $[x]_{\text{原}} = 0.1101$ 求 x

6.1

解：根据定义 $\because [x]_{\text{原}} = 0.1101$

$$\therefore x = +0.1101$$

例 6.4 求 $x = 0$ 的原码

解：设 $x = +0.0000$ $[+0.0000]_{\text{原}} = 0.0000$

$x = -0.0000$ $[-0.0000]_{\text{原}} = 1.0000$

同理，对于整数 $[+0]_{\text{原}} = 0,0000$

$[-0]_{\text{原}} = 1,0000$

$$\therefore [+0]_{\text{原}} \neq [-0]_{\text{原}}$$

2.3.补码表示法

但是用原码作加法时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负

能否 只作加法？

找到一个与负数等价的正数 来代替这个负数

就可使 减 $\longrightarrow$ 加

• 时钟 逆时针 顺时针

$$\begin{array}{r} 6 \\ - 3 \\ \hline 3 \end{array}$$
$$\begin{array}{r} 6 \\ + 9 \\ \hline 15 \end{array}$$

可见 -3 可用 $+9$ 代替 减法 $\longrightarrow$ 加法

$$\begin{array}{r} -12 \\ + 3 \\ \hline \end{array}$$

称 $+9$ 是 -3 以 12 为模的 补数

记作 $-3 \equiv +9 \pmod{12}$

同理 $-4 \equiv +8 \pmod{12}$

$-5 \equiv +7 \pmod{12}$

时钟以
12为模

(1) 补的概念

• 时钟

逆时针

$$\begin{array}{r} 6 \\ - 3 \\ \hline 3 \end{array}$$

顺时针

$$\begin{array}{r} 6 \\ + 9 \\ \hline 15 \\ - 12 \\ \hline 3 \end{array}$$

可见 -3 可用 +9 代替 减法 → 加法

称 +9 是 -3 以 12 为模的补数

记作 $-3 \equiv +9 \pmod{12}$

同理 $-4 \equiv +8 \pmod{12}$

$-5 \equiv +7 \pmod{12}$

时钟以
12为模

结论

6.

➤ 一个负数加上“模”即得该负数的补数

➤ 一个正数和一个负数互为补数时
它们绝对值之和即为模数

• 计数器（模 16） $1011 \longrightarrow 0000$?

$$\begin{array}{r} 1011 \\ - 1011 \\ \hline 0000 \end{array}$$

$$\begin{array}{r} 1011 \\ + 0101 \\ \hline 10000 \end{array}$$

可见 -1011 可用 +0101 代替

记作 $-1011 \equiv +0101 \pmod{2^4}$

同理 $-011 \equiv +101 \pmod{2^3}$

$-0.1001 \equiv +1.0111 \pmod{2}$

自然去掉

(2) 正数的补数即为其本身

两个互为补数的数 $-1011 \equiv +0101 \pmod{2^4}$

分别加上模 $+10000$

结果仍互为补数 $+0101 \equiv +10101$

$\therefore +0101 \equiv +0101 \pmod{2^4}$ 丢掉

可见 $+0101 \xrightarrow{?} +0101$

$\quad \quad \quad \xrightarrow{?} -1011$

? $0,0101 \rightarrow +0101$

? $1,0101 \rightarrow -1011$

$2^{4+1} - 1011 = 100000$

$\quad \quad \quad -1011$

$\quad \quad \quad \hline 1,0101$

用逗号将符号位和数值部分隔开

$(\text{mod } 2^{4+1})$

(3) 补码定义

整数

$$[x]_{\text{补}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

x 为真值 n 为整数的位数

如 $x = +1010$ $x = -1011000$

$[x]_{\text{补}} = 0,1010$ $[x]_{\text{补}} = 2^{7+1} + (-1011000)$

$\quad \quad \quad \uparrow$ $\quad \quad \quad = 100000000$

用逗号将符号位和数值部分隔开 $\quad \quad \quad - 1011000$

$\quad \quad \quad \hline 1,0101000$

小数

6.

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

x 为真值

如 $x = +0.1110$

$$[x]_{\text{补}} = 0.1110$$

用 **小数点** 将符号位
和数值部分隔开

$x = -0.1100000$

$$\begin{aligned} [x]_{\text{补}} &= 2 + (-0.1100000) \\ &= 10.0000000 \\ &\quad - 0.1100000 \\ \hline &1.0100000 \end{aligned}$$

(4) 求补码的快捷方式

设 $x = -1010$ 时

$$\begin{aligned} \text{则 } [x]_{\text{补}} &= 2^{4+1} - 1010 = 11111 + 1 - 1010 \\ &= 100000 = 11111 + 1 \\ &\quad - 1010 \\ \hline &= 1,0110 \end{aligned} \quad \begin{aligned} &= 11111 + 1 - 1010 \\ &= 11111 + 1 \\ &\quad - 1010 \\ \hline &= \boxed{10101} + 1 \\ &= 1,0110 \end{aligned}$$

$$\text{又 } [x]_{\text{原}} = \boxed{1,1010}$$

当真值为 **负** 时，补码 可用 **原码除符号位外**
每位取反，末位加 1 求得

(5) 举例

例 6.5 已知 $[x]_{\text{补}} = 0.0001$

求 x

解：由定义得 $x = +0.0001$

例 6.6 已知 $[x]_{\text{补}} = 1.0001$ $[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}}$

求 x

$$[x]_{\text{原}} = 1.1111$$

解：由定义得 $\therefore x = -0.1111$

$$x = [x]_{\text{补}} - 2$$

$$= 1.0001 - 10.0000$$

$$= -0.1111$$

例 6.7 已知 $[x]_{\text{补}} = 1,1110$

6.1

求 x

解：由定义得

$$[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}}$$

$$x = [x]_{\text{补}} - 2^{4+1}$$

$$[x]_{\text{原}} = 1,0010$$

$$= 1,1110 - 100000$$

$$\therefore x = -0010$$

$$= -0010$$

当真值为负时，原码可用补码除符号位外
每位取反，末位加1求得

练习 求下列真值的补码

6.1

真值	$[x]_{\text{补}}$	$[x]_{\text{原}}$
$x = +70 = 1000110$	0, 1000110	0, 1000110
$x = -70 = -1000110$	1, 0111010	1, 1000110
$x = 0.1110$	0.1110	0.1110
$x = -0.1110$	1.0010	1.1110
$x = \boxed{0.0000} \quad [+0]_{\text{补}} = [-0]_{\text{补}} \quad \boxed{0.0000}$	$\boxed{0.0000}$	0.0000
$x = \boxed{-0.0000}$	$\boxed{0.0000}$	1.0000
$x = -1.0000$	1.0000	不能表示

由小数补码定义 $[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$

$$[-1]_{\text{补}} = 2 + x = 10.0000 - 1.0000 = 1.0000$$

2.4. 反码表示法

(1) 定义

整数

$$[x]_{\text{反}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ (2^{n+1} - 1) + x & 0 \geq x > -2^n \pmod{2^{n+1} - 1} \end{cases}$$

x 为真值 n 为整数的位数

如 $x = +1101$ $x = -1101$

$[x]_{\text{反}} = 0, 1101$ $[x]_{\text{反}} = (2^{4+1} - 1) - 1101$

$\uparrow$ $= 11111 - 1101$

用逗号将符号位 $= 1, 0010$

和数值部分隔开

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2 - 2^{-n}) + x & 0 \geq x > -1 \pmod{2 - 2^{-n}} \end{cases}$$

x 为真值 n 为小数的位数

如

$$x = +0.1101$$

$$x = -0.1010$$

$$[x]_{\text{反}} = 0.1101$$

$$[x]_{\text{反}} = (2 - 2^{-4}) - 0.1010$$

$$= 1.1111 - 0.1010$$

$$= 1.0101$$

用 小数点 将符号位

和数值部分隔开

(2) 举例

例6.8 已知 $[x]_{\text{反}} = 0,1110$ 求 x

解: 由定义得 $x = +1110$

例6.9 已知 $[x]_{\text{反}} = 1,1110$ 求 x

解: 由定义得 $x = [x]_{\text{反}} - (2^{4+1} - 1)$
 $= 1,1110 - 11111$
 $= -0001$

例 6.10 求 0 的反码

解: 设 $x = +0.0000$ $[+0.0000]_{\text{反}} = 0.0000$

$x = -0.0000$ $[-0.0000]_{\text{反}} = 1.1111$

同理, 对于整数 $[+0]_{\text{反}} = 0,0000$ $[-0]_{\text{反}} = 1,1111$

$\therefore [+0]_{\text{反}} \neq [-0]_{\text{反}}$

三种机器数的小结

- 最高位为符号位，书写上用“,”（整数）或“.”（小数）将数值部分和符号位隔开
- 对于正数，原码 = 补码 = 反码
- 对于负数，符号位为 1，其数值部分
原码除符号位外每位取反末位加 1 → 补码
原码除符号位外每位取反 → 反码

部分真值范围表

注意补码，因为补码的0只有一种表现方式，所以多个-128

例6.11 设机器数字长为 8 位（其中 1 位为符号位）
对于整数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

二进制代码	无符号数 对应的真值	原码对应的 真值	补码对应的 真值	反码对应的 真值
00000000	0	+0	±0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

例6.12 已知 $[y]_{\text{补}}$ 求 $[-y]_{\text{补}}$

解： 设 $[y]_{\text{补}} = y_0 \cdot y_1 y_2 \cdots y_n$

<I> $[y]_{\text{补}} = 0.y_1 y_2 \cdots y_n$

$[y]_{\text{补}}$ 连同符号位在内， 每位取反， 末位加 1

即得 $[-y]_{\text{补}}$

$$[-y]_{\text{补}} = 1.\overline{y_1} \overline{y_2} \cdots \overline{y_n} + 2^{-n}$$

<II> $[y]_{\text{补}} = 1.y_1 y_2 \cdots y_n$

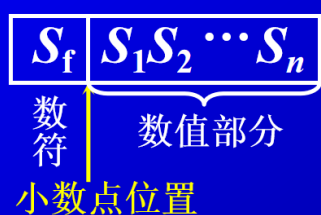
$[y]_{\text{补}}$ 连同符号位在内， 每位取反， 末位加 1

即得 $[-y]_{\text{补}}$

$$[-y]_{\text{补}} = 0.\overline{y_1} \overline{y_2} \cdots \overline{y_n} + 2^{-n}$$

十二、定点表示

注意：n这里表示的是数值部分的位数，不是数据长度的位数，做题时要注意辨别题目信息



或



定点机

小数定点机

整数定点机

原码 $-(1 - 2^{-n}) \sim +(1 - 2^{-n})$ $-(2^n - 1) \sim +(2^n - 1)$

补码 $-1 \sim +(1 - 2^{-n})$ $-2^n \sim +(2^n - 1)$

反码 $-(1 - 2^{-n}) \sim +(1 - 2^{-n})$ $-(2^n - 1) \sim +(2^n - 1)$

十三、算术移位规则

1. 移位的意义

$$15.m = 1500.cm$$

小数点右移 2 位

机器用语 15 相对于小数点 左移 2 位
(小数点不动)

左移 绝对值扩大

右移 绝对值缩小

在计算机中，移位与加减配合，能够实现乘除运算

重点关注补码，按老师说的记忆方法就是，补码是反码末位+1

2. 算术移位规则

符号位不变

真值	码 制	添补代码
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移 添 0 右移 添 1
	反 码	1

十四、原码两位乘（经常考计算题）（必须掌握）

这部分初学比较难懂，建议结合这个视频看：

乘法运算

这里看一下乘法运算，如下，自己理解

$$A = -0.1101 \quad B = 0.1011$$
$$A \times B = -0.10001111 \quad \text{乘积的符号心算求得}$$

0.1101	
$\times 0.1011$	
1101	
1101	
0000	
1101	
0.10001111	

- ✓ 符号位单独处理
- ✓ 乘数的某一位决定是否加被乘数
- ? 4个位积一起相加
- ✓ 乘积的位数扩大一倍

其他补充，相当于前置知识点，不然看的云里雾里的：

原码一位乘

(1) 原码一位乘运算规则

以小数为例

设 $[x]_{\text{原}} = x_0.x_1x_2 \cdots x_n$
 $[y]_{\text{原}} = y_0.y_1y_2 \cdots y_n$

$$[x \cdot y]_{\text{原}} = (x_0 \oplus y_0).(0.x_1x_2 \cdots x_n)(0.y_1y_2 \cdots y_n)$$
$$= (x_0 \oplus y_0).x^*y^*$$

式中 $x^* = 0.x_1x_2 \cdots x_n$ 为 x 的绝对值
 $y^* = 0.y_1y_2 \cdots y_n$ 为 y 的绝对值

乘积的符号位单独处理 $x_0 \oplus y_0$
数值部分为绝对值相乘 $x^* \cdot y^*$

注意 n 为小数点后几位，这和下面的具体需要移位多少次有关

(2) 原码一位乘递推公式

$$\begin{aligned}
 x^* \cdot y^* &= x^* (0.y_1y_2 \dots y_n) \\
 &= x^* (y_12^{-1} + y_22^{-2} + \dots + y_n2^{-n}) \\
 &= 2^{-1} (y_1x^* + 2^{-1}(y_2x^* + \dots 2^{-1}(y_nx^* + 0) \dots))
 \end{aligned}$$

$\underbrace{\qquad \qquad \qquad}_{z_0}$
 $\underbrace{\qquad \qquad \qquad}_{z_1}$
 $\underbrace{\qquad \qquad \qquad}_{z_n}$

$$z_0 = 0$$

$$z_1 = 2^{-1}(y_nx^* + z_0)$$

$$z_2 = 2^{-1}(y_{n-1}x^* + z_1)$$

⋮

$$z_n = 2^{-1}(y_1x^* + z_{n-1})$$

(3) 例题

例6.21 已知 $x = -0.1110$ $y = 0.1101$ 求 $[x \cdot y]_{\text{原}}$ **6.3**

解：

数值部分的运算

部分积	乘数	说明
0.0000	<u>1101</u>	部分积 初态 $z_0 = 0$
+ 0.1110		+ x^*
0.1110		
逻辑右移 0.0111	<u>0110</u>	→ 1, 得 z_1
+ 0.0000		+ 0
0.0111	0	
逻辑右移 0.0011	<u>1011</u>	→ 1, 得 z_2
+ 0.1110		+ x^*
1.0001	10	
逻辑右移 0.1000	<u>1101</u>	→ 1, 得 z_3
+ 0.1110		+ x^*
1.0110	110	
逻辑右移 0.1011	<u>0110</u>	→ 1, 得 z_4

例6.21 结果

① 乘积的符号位 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

② 数值部分按绝对值相乘

$$x^* \cdot y^* = 0.10110110$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.10110110$$

特点 绝对值运算

用移位的次数判断乘法是否结束

逻辑移位

1.概念

原码乘 符号位 和 数值位 部分 分开运算

两位乘 每次用 乘数的 2 位判断 原部分积 是否加 和 如何加 被乘数

乘数 $y_{n-1} y_n$	新的部分积
0 0	加 “0” $\rightarrow 2$
0 1	加 1 倍的被乘数 $\rightarrow 2$
1 0	加 2 倍的被乘数 $\rightarrow 2$
1 1	加 3 倍的被乘数 $\rightarrow 2$

$$\begin{array}{r} 3? \quad 4 \quad 100 \\ \quad -1 \quad -01 \\ \hline \quad 3 \quad 11 \end{array}$$

先 减 1 倍 的被乘数
再 加 4 倍 的被乘数

2.原码两位乘运算规则（必背）

乘数判断位 $y_{n-1}y_n$	标志位 C_j	操作内容
0 0	0	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持“0”
0 1	0	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持“0”
1 0	0	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持“0”
1 1	0	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置“1”
0 0	1	$z+x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置“0”
0 1	1	$z+2x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 置“0”
1 0	1	$z-x^* \rightarrow 2, y^* \rightarrow 2, C_j$ 保持“1”
1 1	1	$z \rightarrow 2, y^* \rightarrow 2, C_j$ 保持“1”

共有操作 $+x^*$ $+2x^*$ $-x^*$ $\rightarrow 2$

实际操作 $+ [x^*]_{\text{补}}$ $+ [2x^*]_{\text{补}}$ $+ [-x^*]_{\text{补}}$ $\rightarrow 2$ 补码移

3.例题

例6.22 已知 $x = 0.111111$ $y = -0.111001$ 求 $[x \cdot y]_{\text{原}}$ **6.3**

解:

部分积	乘数	C_j	说明
补码右移 000.000000 $+ 000.111111$	<u>00.111001</u>	<u>0</u>	初态 $z_0 = 0$ $+x^*, C_j = 0$
000.111111 $+ 000.001111$	11 <u>001110</u>	<u>0</u>	$\rightarrow 2$ $+2x^*, C_j = 0$
补码右移 010.001101 $+ 000.100011$ $+ 111.000001$	11 0111 <u>0011</u>	<u>0</u>	$\rightarrow 2$ $-x^*, C_j = 1$
补码右移 111.100100 $+ 111.111001$ $+ 000.111111$	0111 000111 <u>00</u>	<u>1</u>	$\rightarrow 2$ $+x^*, C_j = 0$
000.111000	000111		

例6.22 结果

① 乘积的符号位 $x_0 \oplus y_0 = 0 \oplus 1 = 1$

② 数值部分的运算

$$x^* \cdot y^* = 0.111000000111$$

$$\text{则 } [x \cdot y]_{\text{原}} = 1.111000000111$$

特点 绝对值的补码运算

用移位的次数判断乘法是否结束

算术移位

4.原码两位乘和原码一位乘比较

	原码一位乘	原码两位乘
符号位	$x_0 \oplus y_0$	$x_0 \oplus y_0$
操作数	绝对值	绝对值的补码
移位	逻辑右移	算术右移
移位次数	n	$\frac{n}{2}$ (n 为偶数)
最多加法次数	n	$\frac{n}{2} + 1$ (n 为偶数)
思考 n 为奇数时, 原码两位乘 移? 次 最多加? 次		

不难理解,当乘数为偶数时,需做 $n/2$ 次移位,最多做 $n/2+1$ 次加法。当乘数为奇数时,乘数高位前可只增加一个“0”,此时需做 $n/2+1$ 次移位(最后一步移一位),最多需做 $n/2+1$ 次加法。

十五、不恢复余数法（加减交替法）（经常考计算题）（必须掌握）

建议结合这个视频看：

https://www.bilibili.com/video/BV1Xe411z7TE/?vd_source=b20f0d3d384c690cf5255b8bf5eb955c

除法运算

1. 分析笔算除法

$$x = -0.1011 \quad y = 0.1101 \quad \text{求 } x \div y$$

		0.1101	
0.1101	∞	0.10110	✓商符单独处理
		0.01101	? 心算上商
		0.010010	? 余数不动低位补“0”
		0.001101	减右移一位的除数
		0.00010100	
		0.00001101	? 上商位置不固定
		0.00000111	

$$x \div y = -0.1101 \quad \text{商符心算求得}$$

余数 0.00000111

2. 笔算除法和机器除法的比较

6.3

笔算除法

商符单独处理

心算上商

余数 不动 低位补“0”
减右移一位的除数

2 倍字长加法器

上商位置 不固定

机器除法

符号位异或形成

$|x| - |y| > 0$ 上商 1

$|x| - |y| < 0$ 上商 0

余数 左移一位 低位补“0”
减 除数

1 倍字长加法器

在寄存器 最末位上商

依旧是前置知识点：

原码除法

以小数为例

$$[x]_{\text{原}} = x_0.x_1x_2 \dots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2 \dots y_n$$

$$\left[\frac{x}{y}\right]_{\text{原}} = (x_0 \oplus y_0) \cdot \frac{x^*}{y^*}$$

式中 $x^* = 0.x_1x_2 \dots x_n$ 为 x 的绝对值
 $y^* = 0.y_1y_2 \dots y_n$ 为 y 的绝对值

商的符号位单独处理 $x_0 \oplus y_0$

数值部分为绝对值相除 $\frac{x^*}{y^*}$

约定 小数定点除法 $x^* < y^*$ 整数定点除法 $x^* > y^*$
被除数不等于 0
除数不能为 0

不恢复余数法（加减交替法）

• 恢复余数法运算规则

余数 $R_i > 0$ 上商 “1”， $2R_i - y^*$

余数 $R_i < 0$ 上商 “0”， $R_i + y^*$ 恢复余数

$$2(R_i + y^*) - y^* = 2R_i + y^*$$

• 不恢复余数法运算规则

上商 “1” $2R_i - y^*$

上商 “0” $2R_i + y^*$

加减交替

例题

例6.25 $x = -0.1011$ $y = -0.1101$ 求 $[\frac{x}{y}]_{\text{原}}$ 6.3

解:	0.1011	0.0000		$[x]_{\text{原}} = 1.1011$
	+1.0011		$+[-y^*]_{\text{补}}$	$[y]_{\text{原}} = 1.1101$
逻辑左移	1.1110	0	余数为负, 上商 0	$[x^*]_{\text{补}} = 0.1011$
	1.1100	0	$\leftarrow 1$	$[y^*]_{\text{补}} = 0.1101$
逻辑左移	+0.1101		$+ [y^*]_{\text{补}}$	$[-y^*]_{\text{补}} = 1.0011$
	0.1001	01	余数为正, 上商 1	
	1.0010	01	$\leftarrow 1$	
逻辑左移	+1.0011		$+ [-y^*]_{\text{补}}$	
	0.0101	011	余数为正, 上商 1	
	0.1010	011	$\leftarrow 1$	
逻辑左移	+1.0011		$+ [-y^*]_{\text{补}}$	
	1.1101	0110	余数为负, 上商 0	
	1.1010	0110	$\leftarrow 1$	
逻辑左移	+0.1101		$+ [y^*]_{\text{补}}$	
	0.0111	01101	余数为正, 上商 1	

例6.25 结果

① $x_0 \oplus y_0 = 1 \oplus 1 = 0$

② $\frac{x^*}{y^*} = 0.1101$

$\therefore [\frac{x}{y}]_{\text{原}} = 0.1101$

特点 上商 $n+1$ 次

第一次上商判溢出

移 n 次, 加 $n+1$ 次

用移位的次数判断除法是否结束

十六、影响指令流水线性能的因素

- 1.结构相关：不同指令争用同一功能部件产生资源冲突
- 2.数据相关：不同指令因重叠操作，可能改变操作数的 读/写 访问顺序
- 3.控制相关：由转移指令引起

十七、流水线性能(必须掌握计算)

1.吞吐率

单位时间内 流水线所完成指令 或 输出结果 的 数量

设 m 段的流水线各段时间为 Δt

- 最大吞吐率

$$T_{pmax} = \frac{1}{\Delta t}$$

- 实际吞吐率

连续处理 n 条指令的吞吐率为

$$T_p = \frac{n}{m \cdot \Delta t + (n-1) \cdot \Delta t}$$

2.加速比

2. 加速比 S_p

8.3

m 段的流水线的速度与等功能的非流水线的速度之比
设流水线各段时间为 Δt

完成 n 条指令在 m 段流水线上共需

$$T = m \cdot \Delta t + (n-1) \cdot \Delta t$$

完成 n 条指令在等效的非流水线上共需

$$T' = nm \cdot \Delta t$$

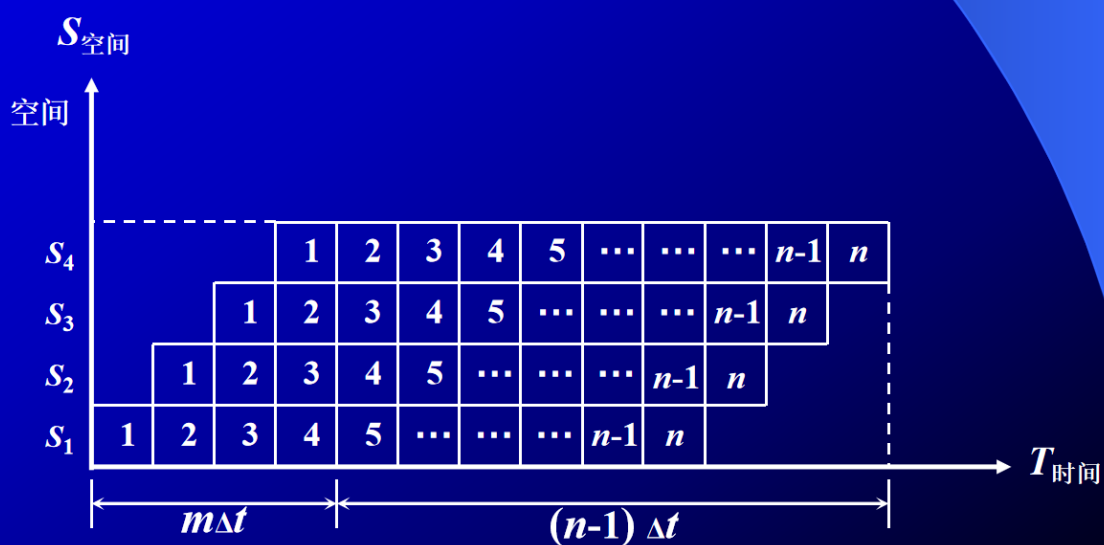
则
$$S_p = \frac{nm \cdot \Delta t}{m \cdot \Delta t + (n-1) \cdot \Delta t} = \frac{nm}{m + n - 1}$$

3. 效率

流水线中各功能段的利用率

由于流水线有建立时间和排空时间

因此各功能段的设备不可能一直处于工作状态



十八、中断隐指令

主要是这个中断隐指令的概念：

在机器指令系统中没有的指令，它是CPU在中断周期内由硬件自动完成的一条指令

(1) 保护程序断点

断点存于 **特定地址**（0 号地址）内 断点 **进栈**

(2) 寻找服务程序入口地址

向量地址 $\rightarrow$ **PC** （硬件向量法）

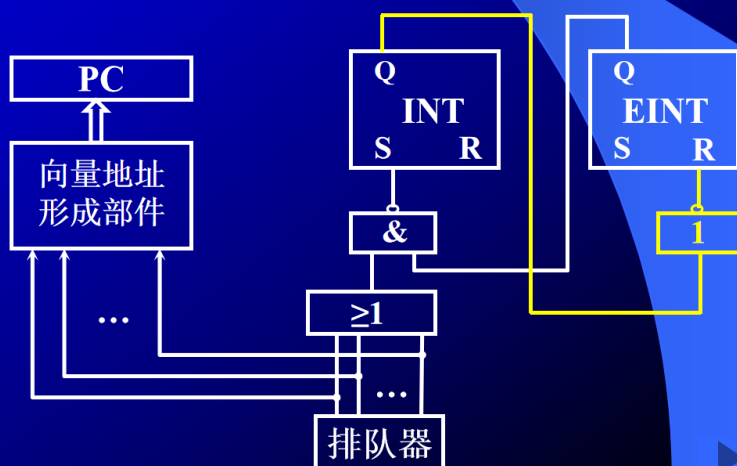
中断识别程序 入口地址 **M** $\rightarrow$ **PC** （软件查询法）

(3) 硬件 关中断

INT 中断标记

EINT 允许中断

R-S 触发器



十九、屏蔽技术(经常出大题)

要求：

学会设置屏蔽字

画图(CPU 执行程序轨迹)：给出特定屏蔽字下，怎么画图

1.屏蔽字

16个中断源 1, 2, 3, … 16 按 降序 排列

优先级	屏蔽字															
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
2	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
3	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
4	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1
5	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1	1
6	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1
⋮	⋮															
15	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
16	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

注：表8.7就是上面表

表 8.7 是对应 16 个中断源的屏蔽字,每个屏蔽字由左向右排序为第 1,2,3…,共 16 位。不难发现,每个中断源对应的屏蔽字是不同的。1 级中断源的屏蔽字是 16 个 1;2 级中断源的屏蔽字是从第 2 位开始共 15 个 1;3 级中断源的屏蔽字是从第 3 位开始共 14 个 1……第 16 级中断源的屏蔽字只有第 16 位为 1,其余各位为 0。

在中断服务程序中设置适当的屏蔽字,能起到对优先级别不同的中断源的屏蔽作用。例如,1 级中断源的请求已被 CPU 响应,若在其中断服务程序中(通常在开中断指令前)设置一个全“1”的屏蔽字,便可保证在执行 1 级中断服务程序过程中,CPU 不再响应任何一个中断源(包括本级在内)的中断请求,即此刻不能实现多重中断。如果在 4 级中断源的服务程序中设置一个

屏蔽字 0001111111111111,由于第 1~3 位为 0,意味着第 1~3 级的中断源未被屏蔽,因此在开中断指令后,比第 4 级中断源级别更高的 1、2、3 级中断源可以中断 4 级中断源的中断服务程序,实现多重中断。

2.屏蔽技术可改变处理优先等级

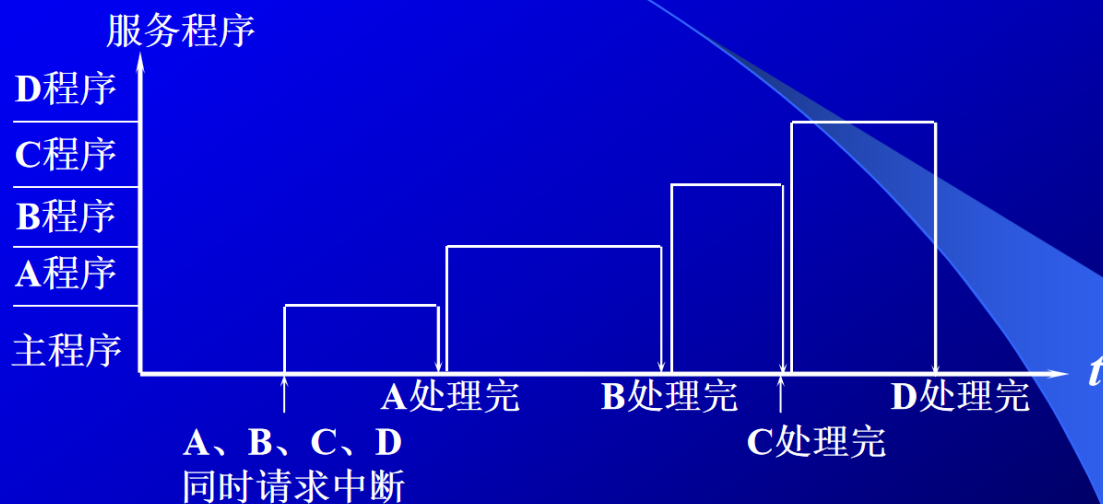
响应优先级 不可改变

处理优先级 可改变（通过重新设置屏蔽字）

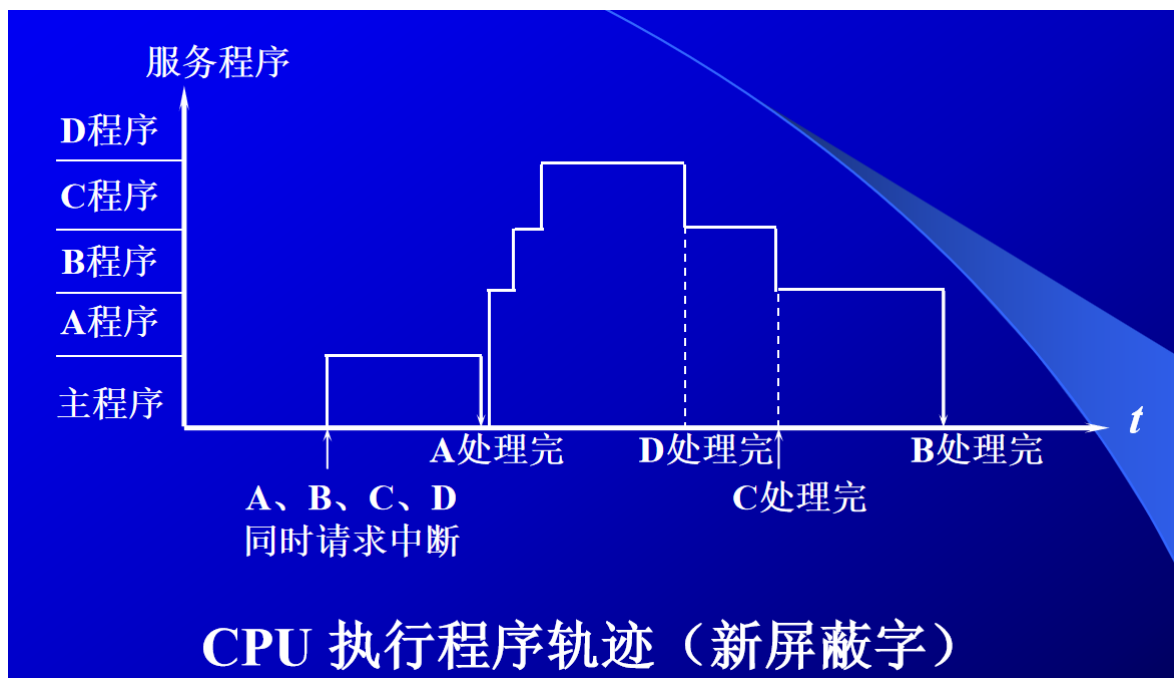
中断源	原屏蔽字	新屏蔽字
A	1 1 1 1	1 1 1 1
B	0 1 1 1	0 1 0 0
C	0 0 1 1	0 1 1 0
D	0 0 0 1	0 1 1 1

响应优先级 **A→B→C→D** 降序排列

处理优先级 **A→D→C→B** 降序排列



CPU 执行程序轨迹（原屏蔽字）



注：图8.35就是上面这个新屏蔽字对应的CPU执行程序轨迹

在同样中断请求的情况下,CPU 执行程序的轨迹发生了变化,如图8.35 所示。CPU 在运行程序的过程中,若 A、B、C、D 4 个中断源同时提出请求,按照中断级别的高低,CPU 首先响应并处理 A 中断源的请求,由于 A 的屏蔽字是 1111,屏蔽了所有的中断源,故 A 程序可以全部执行完,然后回到主程序。由于 B、C、D 的中断请求还未响应,而 B 的响应优先级高于其他,所以 CPU 响应 B 的请求,进入 B 的中断服务程序。在 B 的服务程序中,由于设置了新的屏蔽字 0100,即 A、C、D 可打断 B,而 A 程序已执行完,C 的响应优先级又高于 D,于是 CPU 响应 C,进入 C 的服务程序。在 C 的服务程序中,由于设置了新的屏蔽字 0110,即 A、D 可打断 C,由于 A 程序已执行完,于是 CPU 响应 D,执行 D 的服务程序。在 D 的服务程序中,屏蔽字变成 0111,即只有 A 可打断 D,但 A 已处理结束,所以 D 可以一直执行完,然后回到 C 程序。C 程序执行完后,回到 B 程序。B 程序执行完后,回到主程序。至此,A、B、C、D 均处理完毕。

教材例题：

例 8.2 设某机有 4 个中断源 1、2、3、4,其硬件排队优先次序按 1→2→3→4 降序排列,各中断源的服务程序中所对应的屏蔽字如表 8.9 所示。

表 8.9 例 8.2 各中断源对应的屏蔽字

中断源	屏蔽字			
	1	2	3	4
1	1	1	0	1
2	0	1	0	0
3	1	1	1	1
4	0	1	0	1

(1) 给出上述 4 个中断源的中断处理次序。

(2) 若 4 个中断源同时有中断请求,画出 CPU 执行程序的轨迹。

解:(1) 根据表 8.9,4 个中断源的处理次序是按 3→1→4→2 降序排列。

(2) 当 4 个中断源同时有中断请求时,由于硬件排队的优先次序是 1→2→3→4,故 CPU 先响应 1 的请求,执行 1 的服务程序。由于在该服务程序中设置了屏蔽字 1101,故开中断指令后转去执行 3 的服务程序,且 3 的服务程序执行结束后又回到 1 的服务程序。1 的服务程序结束后,CPU 还有 2、4 两个中断源请求未响应。由于 2 的响应优先级高于 4,故 CPU 先响应 2 的请求,执行 2 的服务程序。在 2 的服务程序中由于设置了屏蔽字 0100,意味着 1、3、4 可中断 2 的服务程序。而 1、3 的请求已处理结束,因此在开中断指令之后转去执行 4 的服务程序,4 的服务程序执行结束后又回到 2 的服务程序的断点处,继续执行 2 的服务程序,直至该程序执行结束。图 8.37 示意了 CPU 执行程序的轨迹。

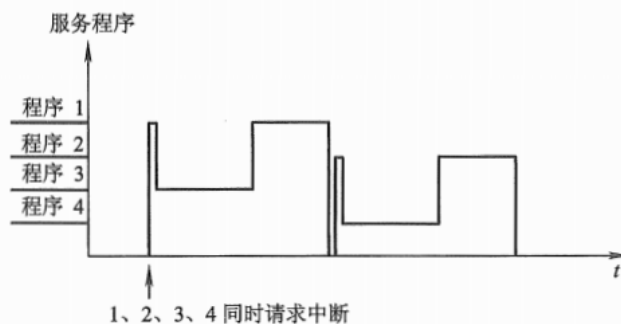


图 8.37 例 8.2 CPU 执行程序的轨迹

二十、机器周期和时钟周期 (节拍、状态)

1. 机器周期

(1) 机器周期的概念

所有指令执行过程中的一个基准时间

(2) 确定机器周期需考虑的因素

每条指令的执行 步骤

每一步骤 所需的 时间

(3) 基准时间的确定

- 以完成 最复杂 指令功能的时间 为准
- 以 访问一次存储器 的时间 为基准

若指令字长 = 存储字长 取指周期 = 机器周期

2.时钟周期（节拍、状态）

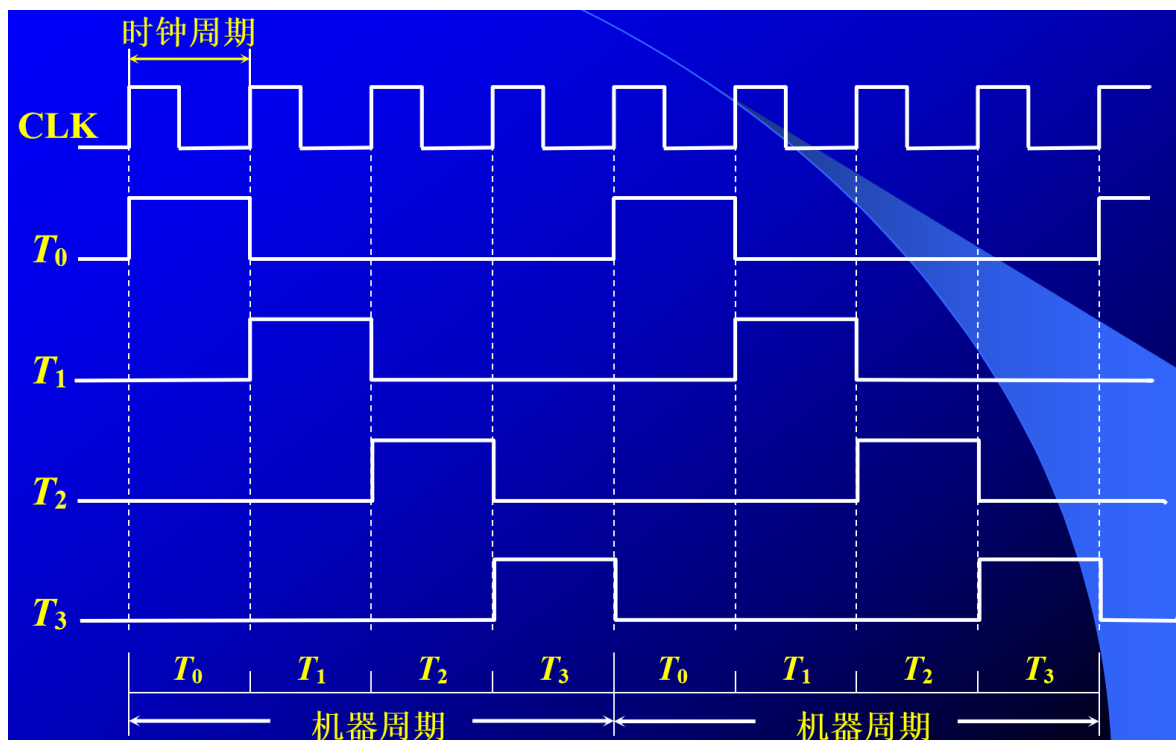
一个机器周期内可完成若干个微操作

每个微操作需一定的时间

将一个机器周期分成若干个时间相等的时间段（节拍、状态、时钟周期）

时钟周期是控制计算机操作的最小单位时间

用时钟周期控制产生一个或几个微操作命令

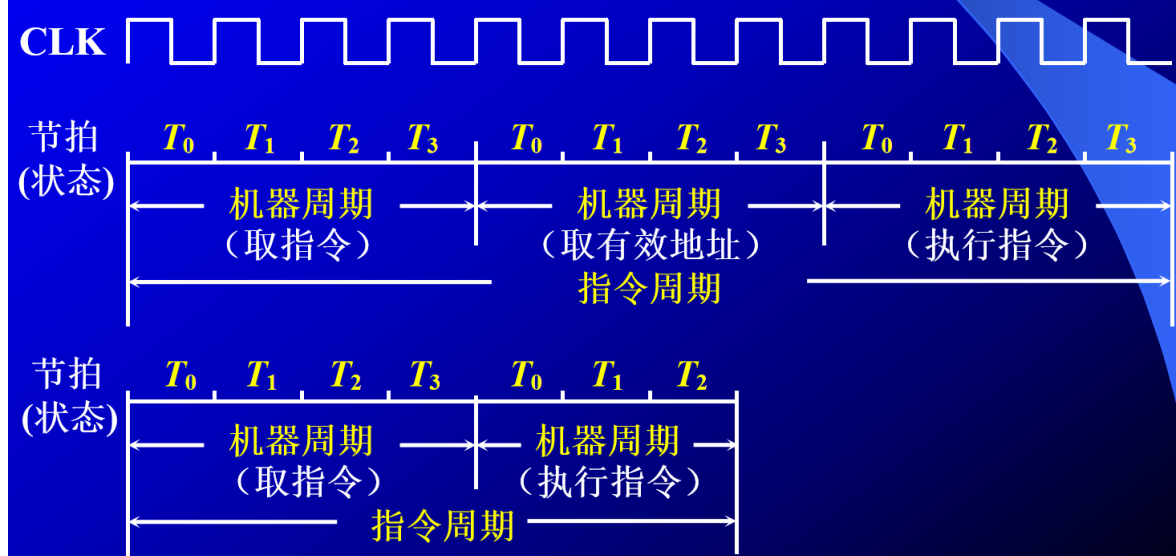


3.多级时序系统

机器周期、节拍（状态）组成多级时序系统

一个指令周期包含若干个机器周期

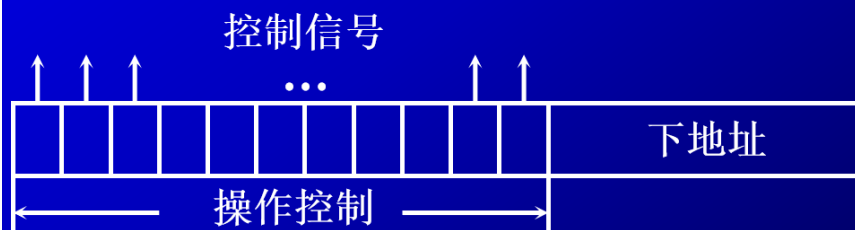
一个机器周期包含若干个时钟周期



二十一、微指令的编码方式（控制方式）

1. 直接编码（直接控制）方式

在微指令的操作控制字段中，
每一位代表一个微操作命令

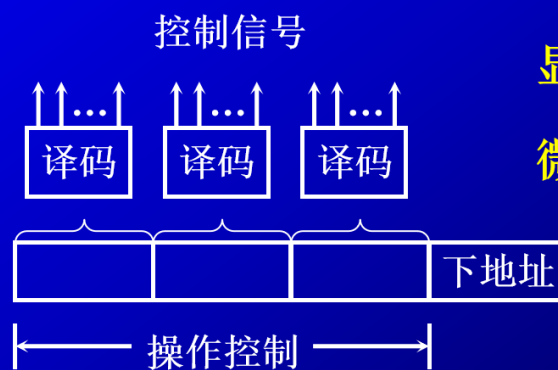


速度最快

某位为“1”表示该控制信号有效

2. 字段直接编码方式

将微指令的控制字段分成若干“段”，
每段经译码后发出控制信号



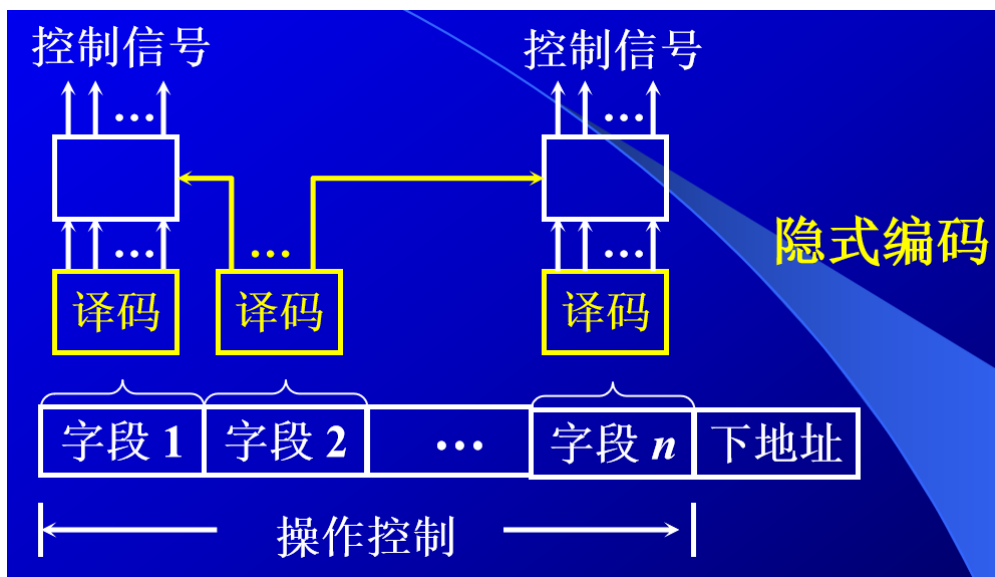
显式编码

微程序执行速度较慢

每个字段中的命令是互斥的

缩短了微指令字长，增加了译码时间

3. 字段间接编码方式



4. 混合编码(记前三个就行)

直接编码和字段编码（直接和间接）混合使用

课本

例10.1和例10.2（经常出大题，必须搞懂）

前置知识点：

10.1.2 微操作的节拍安排

假设机器采用同步控制,每个机器周期包含 3 个节拍,而且 CPU 内部结构如图 9.3 所示,其中 MAR 和 MDR 分别直接和地址总线 and 数据总线相连,并假设 IR 的地址码部分与 MAR 之间有通路。

安排微操作节拍时应注意以下 3 点。

- ① 有些微操作的次序是不容改变的,故安排微操作节拍时必须注意微操作的先后顺序。
- ② 凡是被控制对象不同的微操作,若能在一个节拍内执行,应尽可能安排在同一个节拍内,以节省时间。
- ③ 如果有些微操作所占的时间不长,应该将它们安排在一个节拍内完成,并且允许这些微操作有先后次序。

按上述 3 条原则,以 9.1 节所分析的 10 条指令为例,其微操作的节拍安排如下:

1. 取指周期微操作的节拍安排

- 根据原则②, T_0 节拍可安排两个微操作: $PC \rightarrow MAR, 1 \rightarrow R$ 。
- 根据原则②, T_1 节拍可安排 $M(MAR) \rightarrow MDR$ 和 $(PC)+1 \rightarrow PC$ 两个微操作。
- T_2 节拍可安排 $MDR \rightarrow IR$,考虑到指令译码时间较短,根据原则③,可将指令译码 $OP(IR) \rightarrow ID$ 也安排在 T_2 节拍内。

实际上 $(PC)+1 \rightarrow PC$ 操作也可安排在 T_2 节拍内,因一旦 $PC \rightarrow MAR$ 后,PC 的内容就可修改。

2. 间址周期微操作的节拍安排

T_0 $Ad(IR) \rightarrow MAR, 1 \rightarrow R$
 T_1 $M(MAR) \rightarrow MDR$
 T_2 $MDR \rightarrow Ad(IR)$

3. 执行周期微操作的节拍安排

(1) 非访存指令

1) 清除累加器指令 CLA

该指令在执行周期只有一个微操作,按同步控制的原则,此操作可安排在 $T_0 \sim T_2$ 的任一节拍内,其余节拍空,例如:

T_0
 T_1
 T_2 $0 \rightarrow AC$

2) 累加器取反指令 COM

同理,累加器取反操作可安排在 $T_0 \sim T_2$ 的任一节拍中,即

T_0 T_1 $T_2 \quad \overline{AC} \rightarrow AC$

3) 算术右移一位指令 SHR

 T_0 T_1 $T_2 \quad L(AC) \rightarrow R(AC), AC_0 \rightarrow AC_n$

4) 循环左移一位指令 CSL

 T_0 T_1 $T_2 \quad R(AC) \rightarrow L(AC), AC_0 \rightarrow AC_n \text{ (即 } \rho^{-1}(AC) \text{)}$

5) 停机指令 STP

 T_0 T_1 $T_2 \quad 0 \rightarrow G$

(2) 访存指令

1) 加法指令 ADD X

 $T_0 \quad Ad(IR) \rightarrow MAR, 1 \rightarrow R$ $T_1 \quad M(MAR) \rightarrow MDR$ $T_2 \quad (AC) + (MDR) \rightarrow AC \text{ (该操作实际包括 } (AC) \rightarrow ALU, (MDR) \rightarrow ALU, ALU \rightarrow AC \text{)}$

2) 存数指令 STA X

 $T_0 \quad Ad(IR) \rightarrow MAR, 1 \rightarrow W$ $T_1 \quad AC \rightarrow MDR$ $T_2 \quad MDR \rightarrow M(MAR)$

3) 取数指令 LDA X

 $T_0 \quad Ad(IR) \rightarrow MAR, 1 \rightarrow R$ $T_1 \quad M(MAR) \rightarrow MDR$ $T_2 \quad MDR \rightarrow AC$

(3) 转移类指令

1) 无条件转移指令 JMP X

 T_0 T_1 $T_2 \quad Ad(IR) \rightarrow PC$

2) 有条件转移(负则转)指令 BAN X

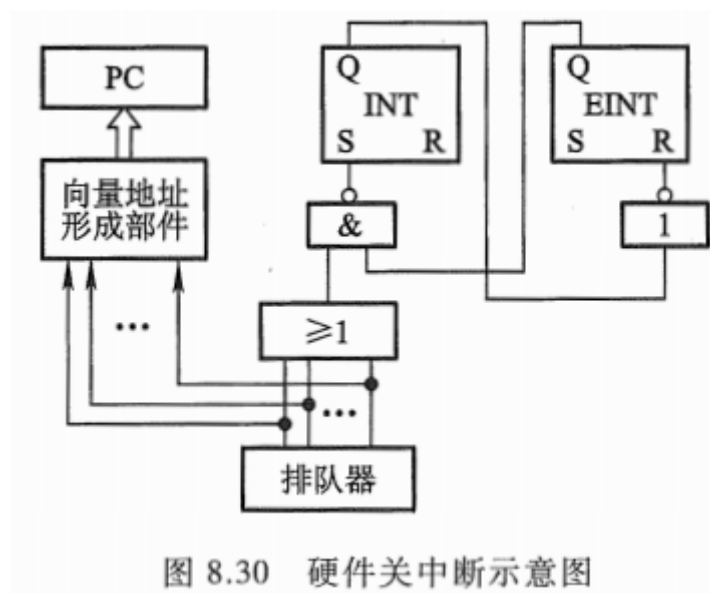
 T_0 T_1 $T_2 \quad A_0 \cdot Ad(IR) + \overline{A_0} \cdot (PC) \rightarrow PC$

4. 中断周期微操作的节拍安排

在执行周期的最后时刻, CPU 要向所有中断源发中断查询信号, 若检测到某个中断源有请求, 并且未被屏蔽又被排队选中, 则在允许中断的条件下, CPU 进入中断周期, 此时 CPU 由中断隐指令完成下列操作(假设程序断点存入主存 0 号地址单元内):

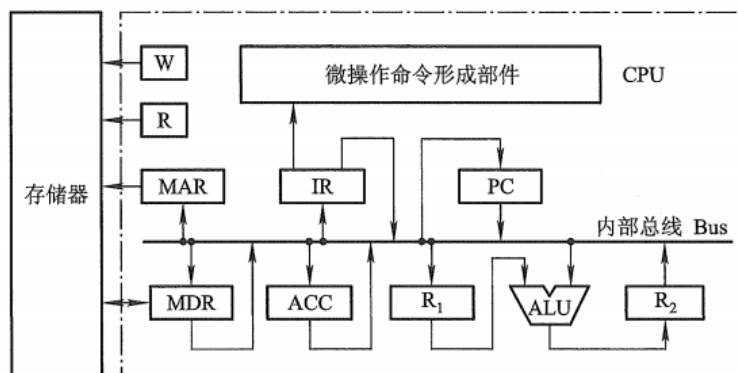
 $T_0 \quad 0 \rightarrow MAR, 1 \rightarrow W$ $T_1 \quad PC \rightarrow MDR$ $T_2 \quad MDR \rightarrow M(MAR), \text{向量地址} \rightarrow PC$

此外, 由图 8.30 可知, CPU 进入中断周期, 由硬件置“0”允许中断触发器 EINT, 即关中断。



例10.1

例 10.1 设 CPU 中各部件及其相互连接关系如图 10.2 所示。图中 W 是写控制标志, R 是读控制标志, R_1 和 R_2 是暂存器。



(1) 假设要求在取指周期由 ALU 完成 $(PC)+1 \rightarrow PC$ 的操作 (即 ALU 可以对它的一个源操作数完成加 1 的运算)。要求以最少的节拍写出取指周期全部微操作命令及节拍安排。

(2) 写出指令“ADD # α ” (#为立即寻址特征, 隐含的操作数在 ACC 中) 在执行阶段所需的微操作命令及节拍安排。

解: (1) 由于 $(PC)+1 \rightarrow PC$ 需由 ALU 完成, 因此 PC 的值可作为 ALU 的一个源操作数, 靠控制 ALU 做 +1 运算得到 $(PC)+1$, 结果送至与 ALU 输出端相连的 R_2 , 然后再送至 PC。

此题的关键是要考虑总线冲突的问题, 故取指周期的微操作命令及节拍安排如下:

T_0 $PC \rightarrow \text{Bus} \rightarrow \text{MAR}, 1 \rightarrow R$; PC 通过总线送 MAR

T_1 $M(\text{MAR}) \rightarrow \text{MDR},$

$(PC) \rightarrow \text{Bus} \rightarrow \text{ALU}, 1 \rightarrow R_2$; PC 通过总线送 ALU 完成 $(PC)+1 \rightarrow R_2$

T_2 $\text{MDR} \rightarrow \text{Bus} \rightarrow \text{IR},$; MDR 通过总线送 IR

$\text{OP}(\text{IR}) \rightarrow \text{微操作命令形成部件}$

T_3 $R_2 \rightarrow \text{Bus} \rightarrow \text{PC}$; R_2 通过总线送 PC

(2) 立即寻址的加法指令执行周期的微操作命令及节拍安排如下:

T_0 $\text{Ad}(\text{IR}) \rightarrow \text{Bus} \rightarrow R_1$; 立即数 $\rightarrow R_1$

T_1 $(\text{ACC}) + (R_1) \rightarrow \text{ALU} \rightarrow R_2$; ACC 通过总线送 ALU

T_2 $R_2 \rightarrow \text{Bus} \rightarrow \text{ACC}$; 结果通过总线送 ACC

例10.2

例 10.2 设 CPU 内部结构如图 10.2 所示,且 PC 有自动加 1 功能。此外还有 B、C、D、E、H、L 6 个寄存器(图中未画),它们各自的输入端和输出端都与内部总线 Bus 相连,并分别受控制信号控制。要求写出完成下列指令组合逻辑控制单元所发出的微操作命令及节拍安排。

- (1) ADD B,C ; $(B) + (C) \rightarrow B$
(2) SUB E,@ H ; $(E) - ((H)) \rightarrow E$ 寄存器间接寻址
(3) STA @ mem ; $ACC \rightarrow ((mem))$ 存储器间接寻址

解:(1) 完成“ADD B,C”指令所需的微操作命令及节拍安排如下:

取指周期

- T_0 PC $\rightarrow$ Bus $\rightarrow$ MAR, 1 $\rightarrow$ R
 T_1 M(MAR) $\rightarrow$ MDR, (PC) + 1 $\rightarrow$ PC
 T_2 MDR $\rightarrow$ Bus $\rightarrow$ IR, OP(IR) $\rightarrow$ 微操作命令形成部件

执行周期

- T_0 C $\rightarrow$ Bus $\rightarrow$ R₁
 T_1 (B) + (R₁) $\rightarrow$ ALU $\rightarrow$ R₂ ; B 通过总线送 ALU
 T_2 R₂ $\rightarrow$ Bus $\rightarrow$ B

(2) 完成“SUB E,@ H”指令所需的微操作命令及节拍安排如下:

取指周期

- T_0 PC $\rightarrow$ Bus $\rightarrow$ MAR, 1 $\rightarrow$ R
 T_1 M(MAR) $\rightarrow$ MDR, (PC) + 1 $\rightarrow$ PC
 T_2 MDR $\rightarrow$ Bus $\rightarrow$ IR, OP(IR) $\rightarrow$ 微操作命令形成部件

间址周期

- T_0 H $\rightarrow$ Bus $\rightarrow$ MAR, 1 $\rightarrow$ R
 T_1 M(MAR) $\rightarrow$ MDR

执行周期

- T_0 MDR $\rightarrow$ Bus $\rightarrow$ R₁
 T_1 (E) - (R₁) $\rightarrow$ ALU $\rightarrow$ R₂ ; E 通过总线送 ALU

- T_2 R₂ $\rightarrow$ Bus $\rightarrow$ E

(3) 完成“STA @ mem”指令所需的微操作命令及节拍安排如下:

取指周期

- T_0 PC $\rightarrow$ Bus $\rightarrow$ MAR, 1 $\rightarrow$ R
 T_1 M(MAR) $\rightarrow$ MDR, (PC) + 1 $\rightarrow$ PC
 T_2 MDR $\rightarrow$ Bus $\rightarrow$ IR, OP(IR) $\rightarrow$ 微操作命令形成部件

间址周期

- T_0 Ad(IR) $\rightarrow$ Bus $\rightarrow$ MAR, 1 $\rightarrow$ R
 T_1 M(MAR) $\rightarrow$ MDR

执行周期

- T_0 MDR $\rightarrow$ Bus $\rightarrow$ MAR, 1 $\rightarrow$ W
 T_1 ACC $\rightarrow$ Bus $\rightarrow$ MDR
 T_2 MDR $\rightarrow$ M(MAR)

其他

一、不同进制之间的转换

十进制、二进制等进制之间的转换

二、其他题目

1、在运算器中，肯定不包含的是（）

A.状态寄存器

B.数据总线

C.ALU

D.地址寄存器

答案：D

2、下列说法有误差的是（）

A.任何二进制整数都可以用十进制表示

B.任何二进制小数都可以用十进制表示

C.任何十进制整数都可以用二进制表示

D.任何十进制小数都可以用二进制表示

答案：D